ESP8266 mit Lazarus programmieren

Vorwort

Vorne weg möchte ich sagen das ich kein Profi auf dem Gebiet des Programmierens bin. Ich mache das ausschließlich als Hobby. Ich kann nicht sagen ob man in diesem Bericht etwas weg lassen oder viel einfacher und besser machen kann. Ich kann nur sagen bei mir Funktioniert es.
Seid also nicht ganz so kritisch. Verbesserungen, Anregungen oder auch einfach nur Meinungen sind sehr willkommen.

Meine Voraussetzungen:
Linux Mint 22.1 Cinnamon (auch in Virtualbox, jedoch nach erfolgtem flashen fror Virtualbox ein. Ich denke Jemand der sich besser auskennt kann das lösen)
Lazarus 4.99 (rev main_4_99-2020-g95205108d4) FPC 3.3.1 x86_64-linux-gtk2
NodeMCU v3.2 - ESP8266 Dev Kit WIFI (von hier: https://www.ebay.de/itm/255283208549)

Viel Spaß beim Ausprobieren und natürlich alles ohne Gewähr!

Lazarus installieren

Meine Quellen:

https://wiki.lazarus.freepascal.org/Xtensa
https://forum.lazarus.freepascal.org/index.php?topic=57725.msg430121#msg430121
https://de.ubunlog.com/pyenv-installiert-mehrere-Python-Versionen-auf-Ihrem-System/
https://github.com/espressif/ESP8266_RTOS_SDK/tree/d412ac601befc4dd024d2d2adcfaa319c7463e36
https://github.com/ccrause/fpc-esp-freertos
https://www.freertos.org/media/2025/FreeRTOS_Reference_Manual_V8.2.1.pdf

Alle orangenen Stellen müssen angepasst werden! Und keine Leerzeichen in den Pfaden verwenden!

Mein System:

Linux Mint 22.1 Cinnamon
FPC 3.3.1 x86_64-linux-gtk2
Lazarus 4.99 (rev main_4_99-2003-g2448ebf956)

FpcUpDeluxe:

FPCUPdeluxe V2.4.0f for x86_64-linux-gtk2
von hier:
https://github.com/newpascal/fpcupdeluxe/releases/latest

Die Version: fpcupdeluxe-x86_64-linux

Installation FpcUpDeluxe:
Zuerst folgende Dateien im Terminal installieren:

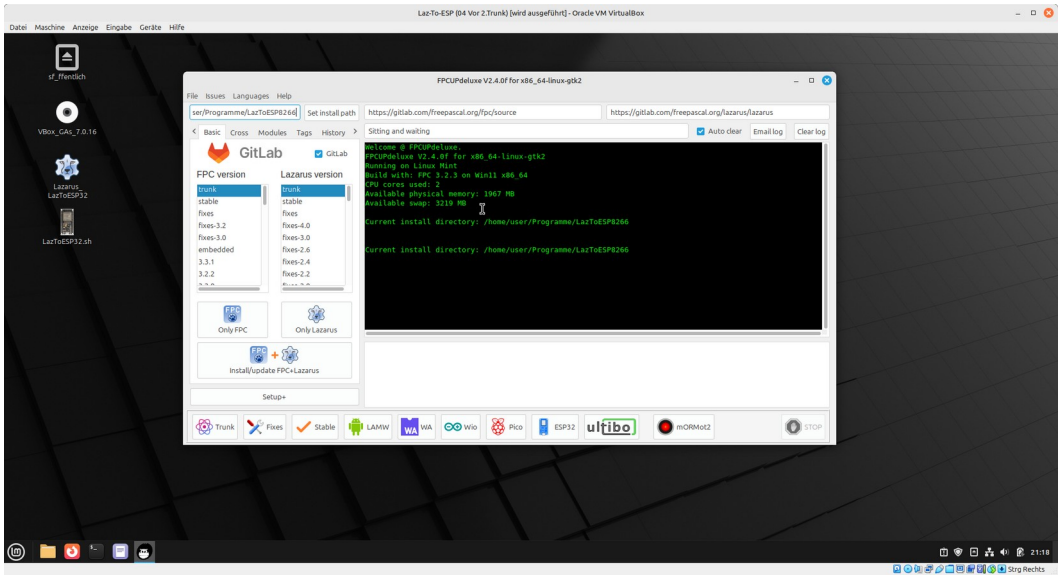
```
sudo apt-get install libx11-dev
sudo apt-get install libgdk-pixbuf2.0-dev
sudo apt-get install libpango1.0-dev
sudo apt-get install libgtk2.0-dev
sudo apt-get install freeglut3-dev
sudo apt-get install git
```

Dann kopiert man sich die Installationsdatei am besten in einen neuen Ordner und klickt diese dort mit der rechten Maustaste an. Dann Eigenschaften und unter Zugriffsrechte bei „Der Datei erlauben sie als Programm auszuführen“ einen Haken machen und schließen.

Lazarus Trunk (jetzt Main) installieren:

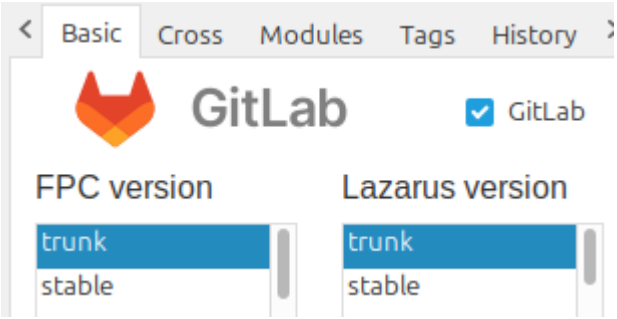
Zuerst legt man einen neuen leeren Ordner an in dem die Installation erfolgen soll. Bei mir /home/user/Programme/LazToESP8266.

Nun FpcUpDeluxe starten.



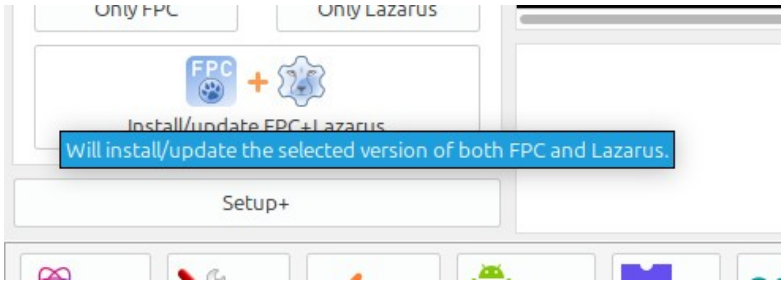
Jetzt auf „Set Install Path“ klicken und zu dem vorher neu angelegten Ordner navigieren.

Nun Trunk und Trunk anwählen:

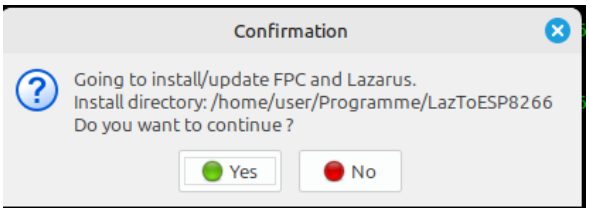


Optional: Auf „Setup+“ klicken. Die Option Be extra verbose liefert mehr Ausgaben im Logfenster und wenn man möchte kann man gleich die Docked Lazarus IDE mit installieren.

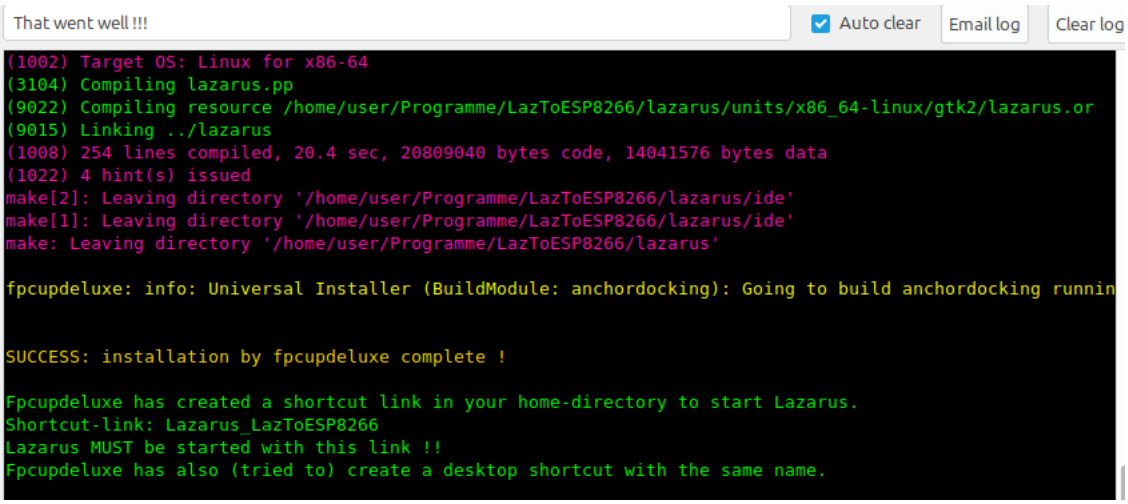
Jetzt „install/update the selected version of both FPC and Lazarus“ klicken:



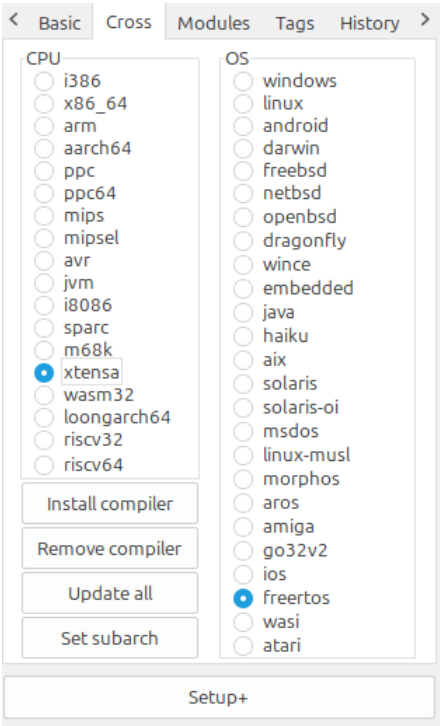
Mit Yes bestätigen:



Jetzt beginnt die Installation und mit etwas Glück läuft sie ohne Probleme durch. Da die Trunk (Main) Versionen nicht stabil sind kann es vorkommen das die Installation mal einige Tage nicht funktioniert. Dann einfach öfter mal probieren.

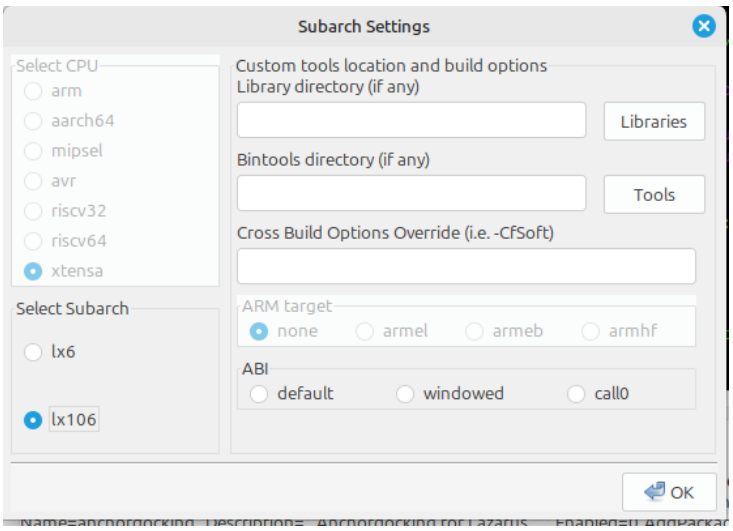


Wenn man möchte kann man Lazarus nun starten. Hat man die Option Dockedform gewählt sollte man nun bei Modern IDE und Modern Form Editor den Haken setzen. Dann „Start IDE“. Zum Umstellen auf Deutsch „Tools, Options, General, German, okay“. IDE schließen und wieder starten. Nun ist alles auf Deutsch eingestellt. Lazarus schließen.

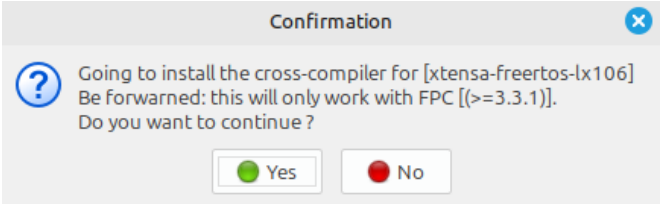


Weiter gehts in FpcUpDeluxe. Hier den Reiter Cross aktivieren. Dort bei CPU extensa und bei OS freertos wählen.

Nun „Set subarch“ klicken. Dort einen Haken bei lx106 machen und Ok.

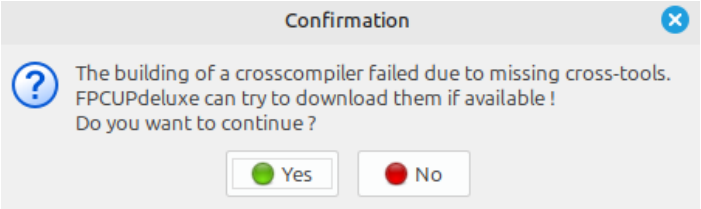


Nun „Install Compiler“ klicken. Es kommt folgendes Fenster:

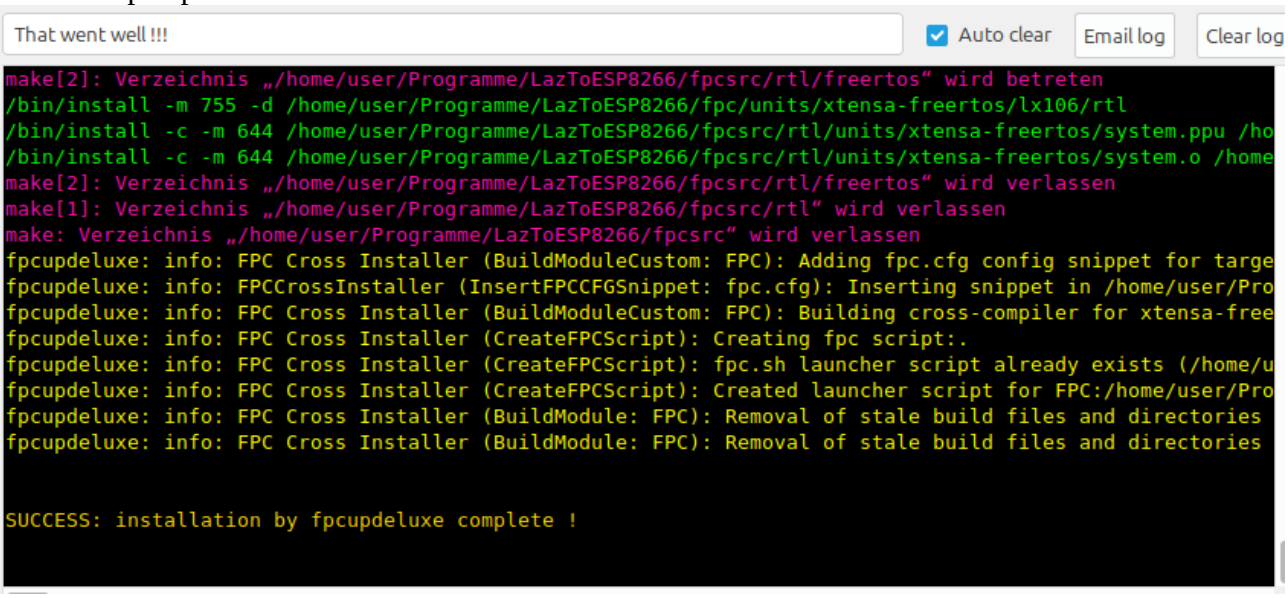


Dies mit Yes bestätigen.

Nun das downloaden der benötigten Dateien bestätigen:



Nun hat FpcUpDeluxe alles installiert:



Anpassen der Python-Version

Unter /home/user/Programme/LazToESP8266/cross/bin/xtensa-freertos/esp-rtos-3.4 findet man eine Datei requirements.txt. Darin stehen die benötigten Phytonpakete bzw. installiert diese Datei die benötigten Packete.
Damit Lazarus mit den benötigten Paketen laufen kann muss man es mit einer passenden Python-Version starten. Dafür muss zunächst eine Virtuelle Umgebung geschaffen werden. Dazu benötigt man das Programm pyenv.

Führen Sie folgende Befehle in Ihrem Terminal aus, um pyenv zu installieren:

```
sudo apt-get install -y make build-essential git libssl-dev zlib1g-dev libbz2-dev libreadline-dev libsqlite3-dev wget curl llvm libncurses5-dev libncursesw5-dev xz-utils tk-dev
```

```
curl -L https://raw.githubusercontent.com/pyenv/pyenv-installer/master/bin/pyenv-installer | bash
```

```
nano ~/.bash_profile
```

Nun die folgenden Zeilen am Ende der Datei hinzu fügen, hier müssen wir "**USER**" durch Ihren Systembenutzernamen **ersetzen**.

```
export PATH="/home/USER/.pyenv/bin:$PATH"
eval "$(pyenv init -)"
eval "$(pyenv virtualenv-init -)"
```

Die Änderungen mit Strg + O, Enter speichern und mit Strg + X nano beenden.
Um pyenv dem System bekannt zu machen muss (immer beim neu Öffnen des Terminals) folgender Befehl eingegeben werden:

```
source ~/.bash_profile
```

Möchte man wissen welche Python-Versionen in unserem System verfügbar sind folgenden Befehl eingeben:

```
pyenv install --list
```

Meine Arduino IDE verwendet 3.7.2. Leider lies sich diese Version bei mir nicht installieren. Ich habe mich deshalb für 3.7.17 entschieden.

Jetzt die Version 3.7.17 installieren:

```
pyenv install    3.7.17
```

Um die Version 3.7.17 von Python zu aktivieren:

```
pyenv global  3.7.17
```

Terminal nicht schließen. Lazarus muss in diesem Zustand aus dem Terminal heraus gestartet werden damit es unter pyenv läuft!!

Hat man Lazarus aus versehen geschlossen bzw. möchte man aus irgend einem Grund Lazarus unter pyenv starten kann man das mit diesen Befehlen tun oder sich gleich eine kleine Start-Bash erstellen:
Einen Texteditor öffnen und folgendes eingeben:

```
#!/bin/bash
cd
source ~/.bash_profile
pyenv global 3.7.17
echo Es ist folgende Python_Version aktiv:
python3 --version
cd /home/user/Programme/LazToESP8266/lazarus
./lazarus
```

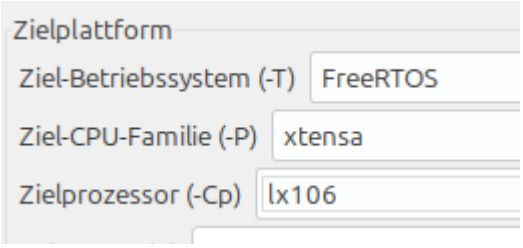
Lazarus anpassen

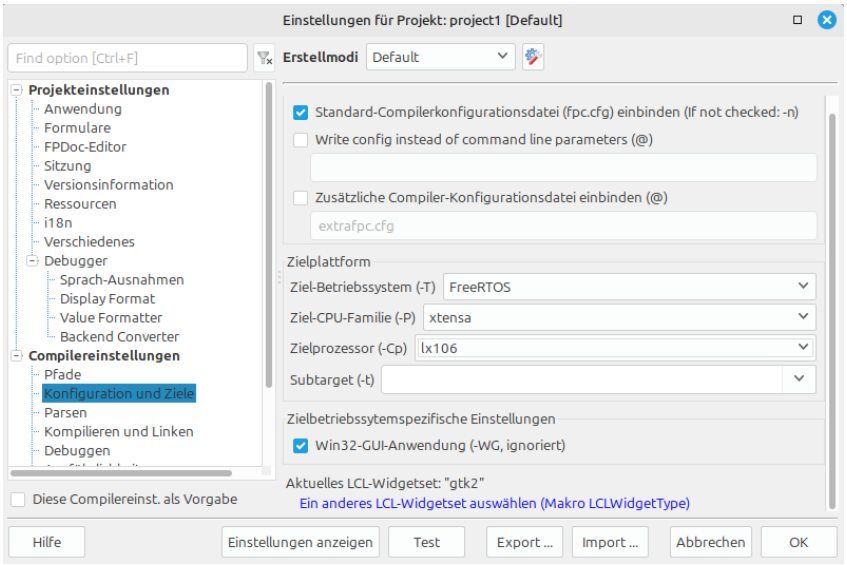
Ins Lazarusverzeichnis wechseln und starten:

```
cd /home/user/Programme/LazToESP8266/lazarus
./lazarus
```

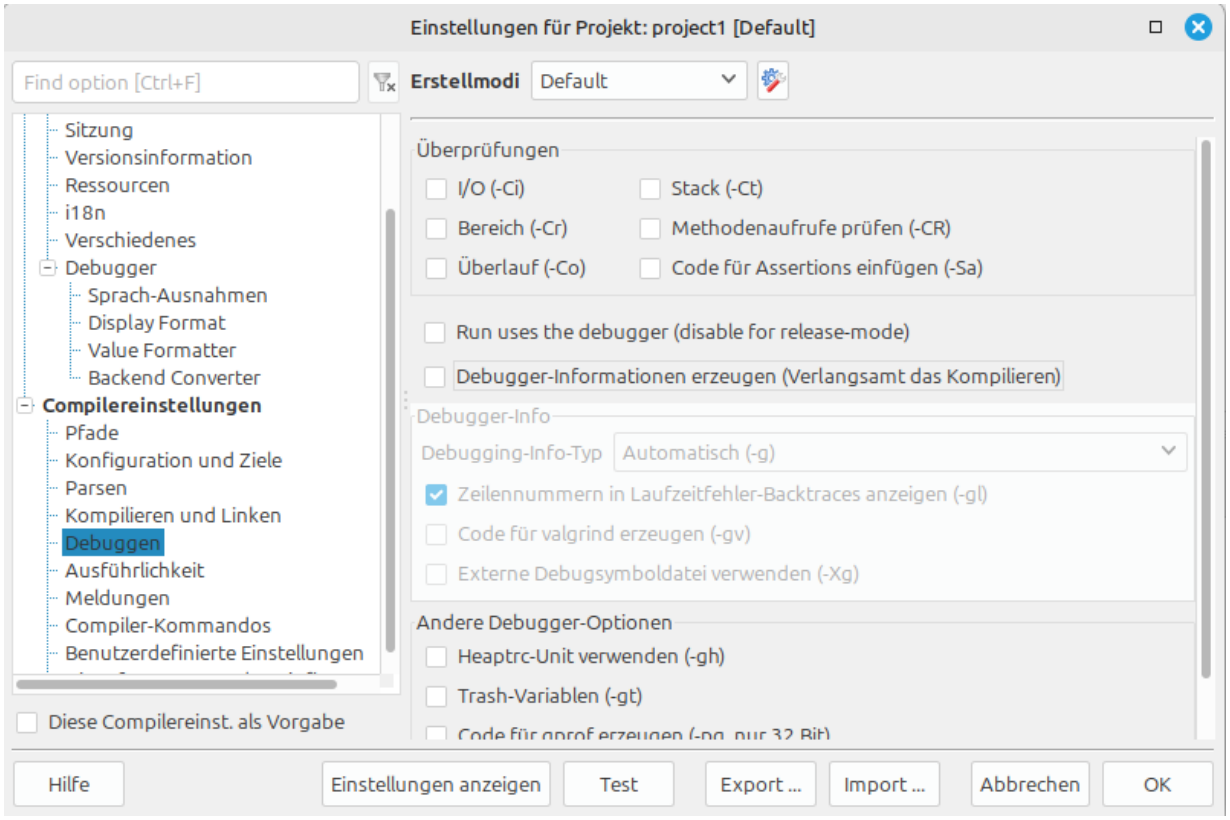
Nachdem Lazarus gestartet ist muss noch der passende Erstellmodi eingestellt werden:
Projekt, Projekteinstellungen, Konfiguration und Ziele

Das hier Einstellen:





Dann in dem Reiter „Debuggen“ und dort alles deaktivieren:



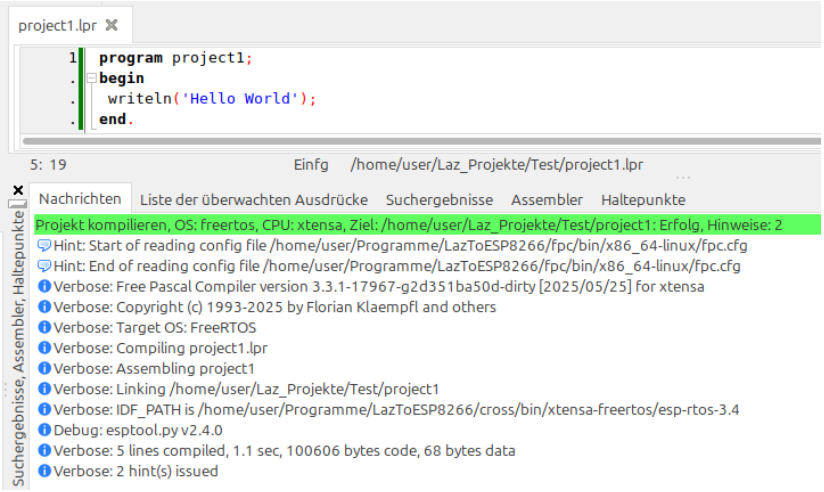
Wenn man möchte das diese Einstellung bei einem neuen Projekt gleich drin steht muss man einen Haken bei „Diese Compilereinst. als Vorgabe“ machen. Mit Ok das Fenster schließen.

Jetzt auf Datei, Neu, Einfaches Programm und folgendes Minimal Projekt eingeben:

```
program project1;
begin
 writeln('Hello World');
end.
```

Das Mini Projekt mit Speichern Unter in ein eigenes Verzeichnis abspeichern. Dann auf Start und Kompillieren (oder Strg+F9).

Ergebnis:



Nach dem ersten Mal kompillieren (**mit pyenv**) befindet sich unter
*/home/user/Programme/LazToESP8266/*cross/bin/xtensa-freertos/esp-rtos-3.4/ der Ordner venv.
Darin findet man nun u. a. die benötigten Packages.
Ab diesem Zeitpunkt kann Lazarus ganz normal über das von FpcUpDeluxe angelegte Starter Icon geöffnet werden.

Flashen

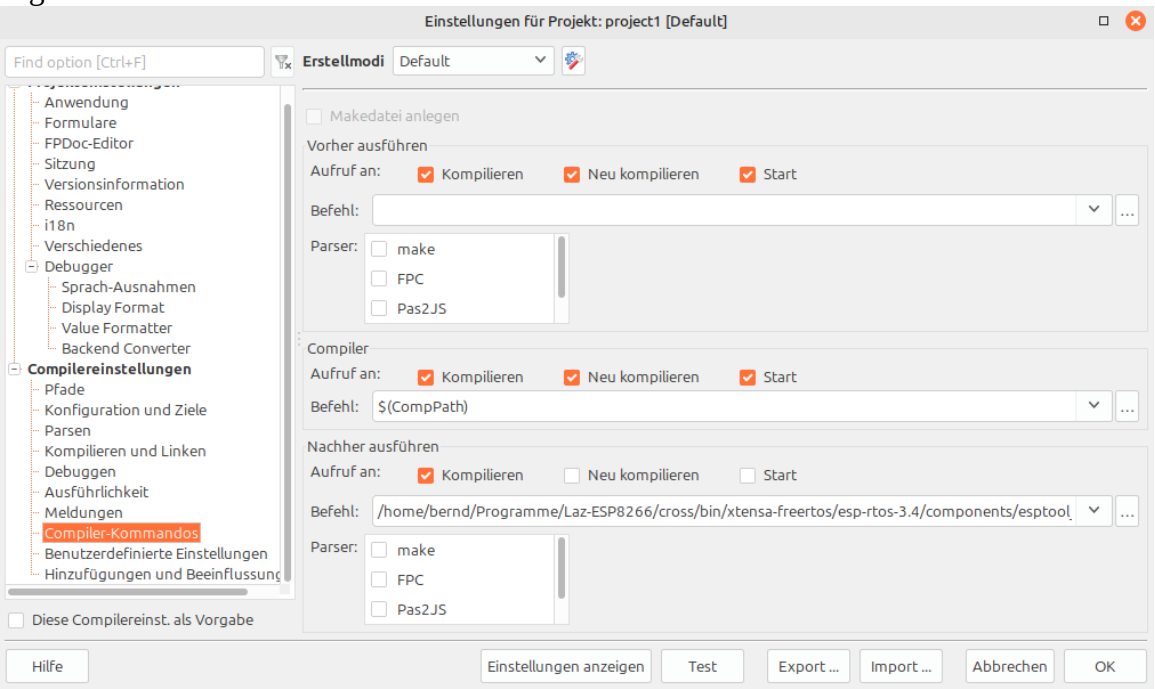
Damit das Kompillierte Programm auf den ESP8266 geflasht wird muss noch folgendes getan werden:

Der Befehl zum Flashen schaut in etwa so aus. Es müssen die Pfade angepasst werde!

/home/user/Programme/LazToESP8266/cross/bin/xtensa-freertos/esp-rtos-3.4/components/esptool_py/esptool/esptool.py --chip esp8266 --port "/dev/ttyUSB0" --baud 115200 --before "default_reset" --after "hard_reset" write_flash -z --flash_mode "dio" --flash_freq "40m" --flash_size "4MB" 0x0 /home/user/Programme/LazToESP8266/cross/lib/xtensa-freertos/lx106/bootloader.bin 0x10000 project1.bin 0x8000 /home/user/Programme/LazToESP8266/cross/lib/xtensa-freertos/lx106/partitions_singleapp.bin

Unter VirtualBox:
PlatformIO [Errno 13] Permission denied: '/dev/ttyUSB0'
ESP8266 anstecken.
sudo chmod a+rw /dev/ttyUSB0

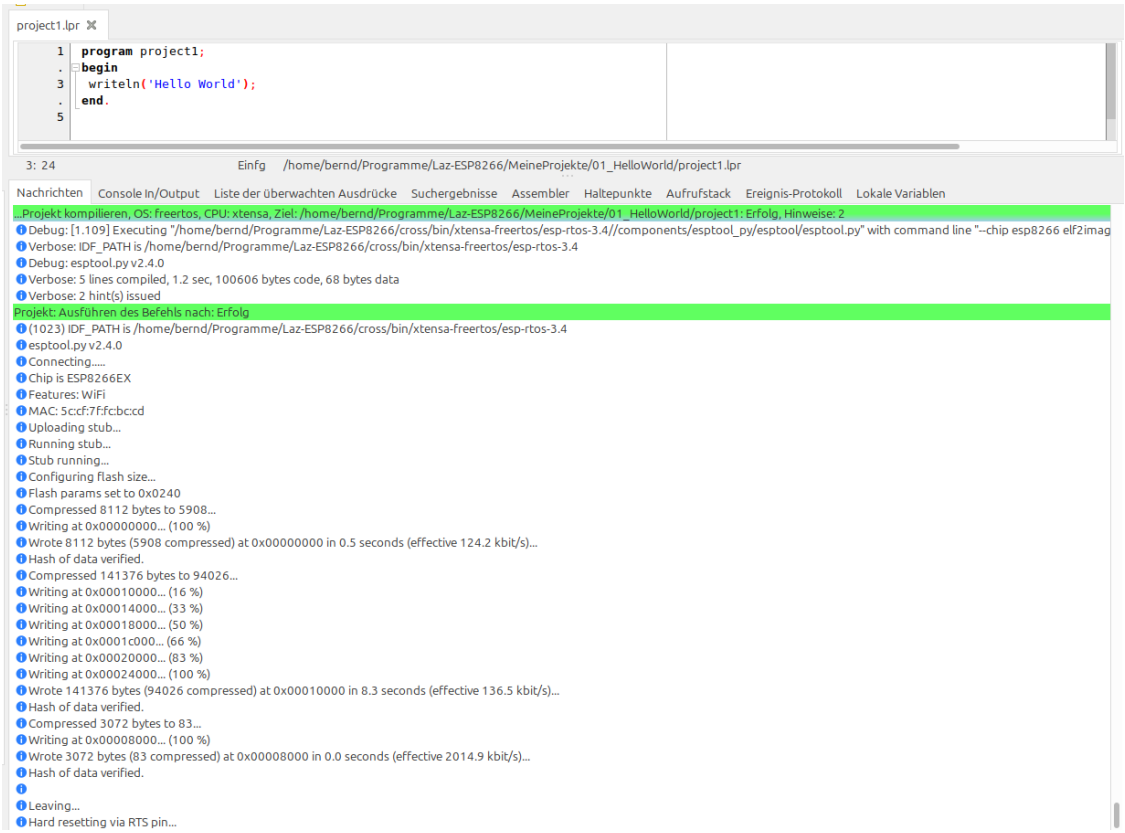
Dieser Befehl muss nun bei Projekt, Projekteinstellungen, Compiler-Kommandos in die Zeile Befehl bei Nachher ausführen kopiert werden. Die Haken bei Neu kompilieren und Start entfernen! Achtung Vorne und Hinten keine Leerzeichen!



Wenn man möchte Haken bei Diese Compilereinst. als Vorgabe. OK.

ESP8266 anstecken und dann kompillieren [Strg+F9]

So sollte es dann in Lazarus aussehen:



Hier der Link zu einem verbessertem Flash Befehl:

Flash Befehl

Tipp: Anpassen der IDE Cool-Bar

Ich habe bei mir die beiden grünen Dreiecke für Start bzw. Start ohne Debugger ausgeblendet.

Und dafür die beiden Zahnräder für Kompilieren und Neu Kompilieren(Build) eingefügt.

Neu Kompilieren (Build) kompilliert das Projekt nur während Kompillieren auch noch flasht.

Wer das möchte Werkzeuge, Einstellungen, Umgebung, IDE Coolbar. Mit Hinzufügen und Löschen einfach alles wie gewünscht zusammen stellen. Falls einem die Hinweise die Angezeigt werden nicht gefallen kann man in der Datei lazaruside.de.po die Übersetzung abändern. Zum Beispiel in Kompiliern und Hochladen bzw. Nur Kompilieren.

Tipp: Farbige Begin/End Blöcke
Wer farbige begin und end (repeat/until etc.) Blöcke haben möchte kann unter Werkzeuge, Einstellungen, Editor,
Anzeige, Hervorhebungen, dann rechts bei Außenlinien (global) einen Haken machen.

Libraries anpassen

Leider konnte ich in späteren Programmen keine Verbindung mit dem Wifi herstellen. Da ich keine Lösung fand habe ich im Internationalen Forum um Hilfe gebeten. Dort hat mir ccrause erklärt das es an den von mir verwendeten Libraries liegt.
https://forum.lazarus.freepascal.org/index.php/topic,72082.0.html

Die von mir verwendeten Libraries wurden von FpcUpDeluxe geladen. Es kann natürlich sein das bei einer späteren Installation andere Libraries geladen werden und dieser Schritt nicht mehr nötig ist! Meine von FpcUpDeluxe geladen Libraries besitzen das Datum 18.Januar.2022.

Die Libraries von ccrause befinden sich hier:
https://github.com/ccrause/fpc-esp-freertos/tree/master/snapshot
Dort benötigt man die fpc-esp8266-idf-3.4-20220107.zip.

Diese zip downloaden und in den Ordner /home/bernd/Programme/Laz-ESP8266/cross/lib/xtensa-freertos entpacken. Dann vorsichtshalber den Ordner lx106 umbenennen und einen leeren neuen Ordner mit den Namen lx106 anlegen. Nun zuerst den gesamten originalen Ordnerinhalt vom alten lx106 in den neuen kopieren. Jetzt alle Libraries (nur die Libraries, also alle libxxx.a!) von ccrause in den neuen lx106 kopieren und überschreiben anklicken.
Die neuen Libraries haben das Datum 6.Januar.2022.
Damit waren meine Probleme mit der Wifi Verbindung behoben.

Weitere Info:
Ab ESP8266 RTOS SDK v3.x wurde eine teilweise Annäherung / API-Anpassung an den ESP32 vorgenommen.

Serielle Ausgabe:

Um das Hello World in der Seriellen Schnittstelle zu Empfangen habe ich das Programm CuteCom benutzt.
Der ESP8266 läuft während des Bootmodus immer mit der Baudrate 74880. Wird im Programm nichts anderes gesetzt läuft das Programm entweder mit 74880 oder 115200 Baud. Es kommt darauf an welche SDK Libraries verwendet wurden. Nimmt man die Libraries von ccrause (wie oben beschrieben) läuft das Programm mit 115200 Baud.
https://forum.lazarus.freepascal.org/index.php?topic=72082.msg563769#msg563769

```
08:52:43:810
08:52:43:810 ets Jan  8 2013,rst cause:2, boot mode:(3,6)
08:52:43:812
08:52:43:821 load 0x40100000, len 5504, room 16
08:52:43:832 tail 0
08:52:43:832 chksum 0x07
08:52:43:832 load 0x3ffe8408, len 24, room 8
08:52:43:837 tail 0
08:52:43:837 chksum 0x8e
08:52:43:841 load 0x3ffe8420, len 2548, room 8
08:52:43:850 tail 12
08:52:43:850 chksum 0xec
08:52:43:850 csum 0xec
08:52:43:937 Hello World
08:52:43:937
08:52:43:937 _haltproc called, exit code: 0
```

Die Zeilen vor Hello World werden im Bootmodus ausgegeben und sind nur bei einer Baudrate von 74880 sichtbar.

Hinweis: 74880 lässt sich unter Linux nicht so ohne weiteres einstellen! Ich weiß noch nicht wie CuteCom oder die Arduino IDE das machen. Stellt man bei Synaser diese Baudrate ein kommt keine Fehlermeldung aber auch keine Ausgabe. Ich hab das ziemlich lange nicht bemerkt.

Zeigt unter Linux im Terminal die Einstellugen der Seriellen:

```
stty -a < /dev/ttyUSB0
```

Der Befehl sollte die Baudrate einstellen, bringt aber eine Fehlermeldung:

```
stty -F /dev/ttyUSB0 74880
```

Programmieren

Das erste Blinker Programm

Das Programm hab ich hier gefunden:
https://forum.lazarus.freepascal.org/index.php?topic=57725.msg430121#msg430121

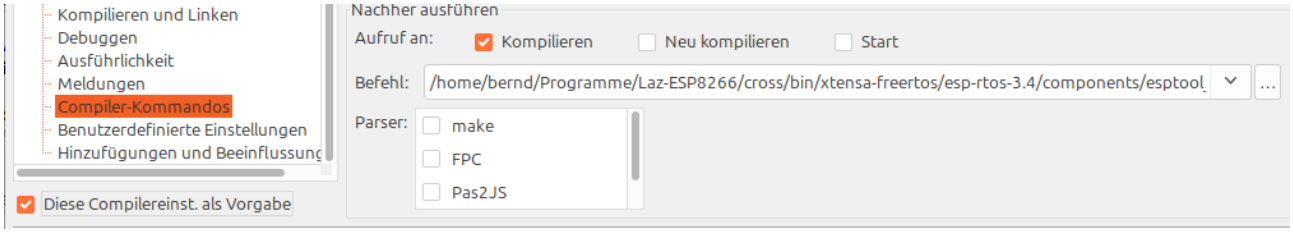
Lazarus ganz normal öffnen. Datei, Neu, Einfaches Programm, Ok.
Diesen Quelltext eingeben:

```
program project1;
type
  Tgpio_config = record
    pin_bit_mask: uint32;  // Pin mask of pins to configure
    mode: integer;         // 0 = disabled, 1 = input, 2 = output, 6 = output open drain
    pull_up_en: integer;   // 0 = disabled, 1 = enabled
    pull_down_en: integer; // 0 = disabled, 1 = enabled
    intr_type: integer;    // 0 = disabled, 1 = positive edge, 2 = negative edge, 3 = any edge, 4 =
low level, 5 = high level
  end;

const
  // GPIO pin number for pin connected to LED
  LED = 2;  // NodeMCU LED on ESP-12E module

// Return value is error code, 0 = success
function gpio_config(constref gpio_cfg: Tgpio_config): integer; external;

// mode: GPIO_MODE_DISABLE = 0, GPIO_MODE_INPUT = 1, GPIO_MODE_OUTPUT = 2, GPIO_MODE_OUTPUT_OD = 6
function gpio_set_direction(gpio_num: integer; mode: integer): integer;
  external;

// level: Low = 0, High = 1
function gpio_set_level(gpio_num: integer; level: uint32): integer; external;

procedure vTaskDelay(xTicksToDelay: uint32); external;

var
  cfg: Tgpio_config;

begin
  cfg.pin_bit_mask := 1 shl ord(LED);
  cfg.mode := 1;
  cfg.pull_up_en := 0;
  cfg.pull_down_en := 0;
  cfg.intr_type := 0;
  gpio_config(cfg);

  gpio_set_direction(LED, 2);
  repeat
    writeln('.');
    gpio_set_level(LED, 0);
    vTaskDelay(10);
    writeln('*');
    gpio_set_level(LED, 1);
    vTaskDelay(10);
  until false;
end.
```

Das Projekt in einem eigenen Verzeichnis abspeichern.

Wenn beim ersten Projekt „Hello World“ der Haken „Diese Compilereinst. als Vorgabe“ gesetzt wurde hat Lazarus diese Einstellungen ins neue Projekt übernommen. Ansonsten alle Einstellungen wie oben beschrieben neu eingeben. Da im Flash Befehl der Projektname bisher hart Codiert wurde sollte man dafür besser das IDE Makro TargetFile verwenden. Man muss dann den Befehl nicht jedesmal anpassen:

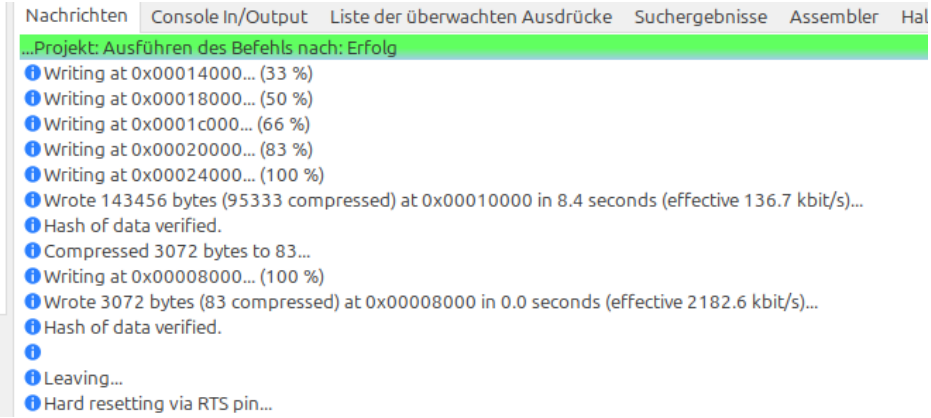
/home/bernd/Programme/Laz-ESP8266/cross/bin/xtensa-freertos/esp-rtos-3.4/components/esptool_py/esptool/esptool.py --chip esp8266 --port "/dev/ttyUSB0" --baud 115200 --before "default_reset" --after "hard_reset" write_flash -z --flash_mode "dio" --flash_freq "40m" --flash_size "4MB" 0x0 /home/bernd/Programme/Laz-ESP8266/cross/lib/xtensa-freertos/lx106/bootloader.bin 0x10000 $(TargetFile)**.bin** 0x8000 /home/bernd/Programme/Laz-ESP8266/cross/lib/xtensa-freertos/lx106/partitions_singleapp.bin



Wenn man möchte Haken bei „Diese Compilereinst. als Vorgabe“

ESP8266 anstecken und dann kompillieren [Strg+F9]

So sollte es dann in Lazarus aussehen:



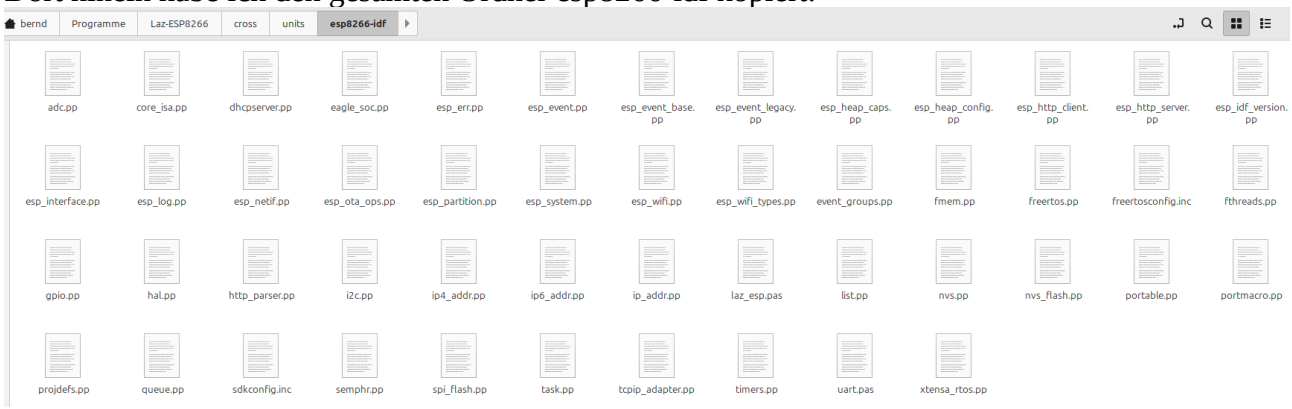
Die integrierte blaue LED des ESP8266 sollte nun blinken!

Bindings von ccrause

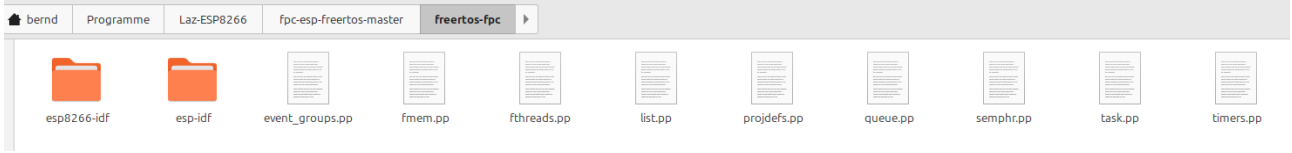
Um den ESP8266 richtig nutzen zu können benötigt man aber noch die passenden Pascal Header (Bindings). Auf der GitHub Seite von ccrause findet man diese. Da er sie unter die Mozilla Public License Version 2.0 gestellt hat sollte es meiner Meinung nach kein Problem sein diese zu benutzen. Also am besten hierhin:
https://github.com/ccrause/fpc-esp-freertos/tree/master
springen und das ganze Repo Klonen oder als zip downloaden **und einen Stern dort lassen**.
Direkter Download:
https://github.com/ccrause/fpc-esp-freertos/archive/refs/heads/master.zip

Dann im Lazarus-Verzeichnis einen neuen Ordner mit dem Namen units anlegen. Ich habe es unter home/bernd/Programme/Laz-ESP8266/cross/units gemacht.

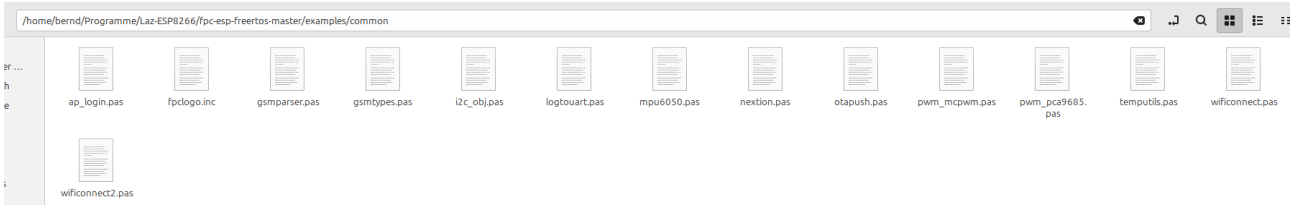
Dort hinein habe ich den gesamten Ordner esp8266-idf kopiert:



Zusätzlich habe ich noch die 9 Dateien aus dem Ordner freeertos-fpc hineinkopiert:



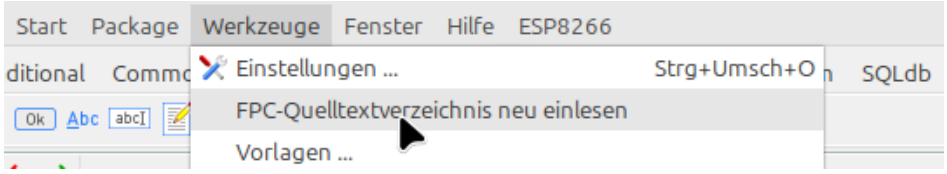
und noch alles aus fpc-esp-freertos-master/examples/common



Dann die Datei fpc.cfg öffnen und an der Stelle den Suchpfad für units ergänzen:
(Mein Pfad zur fpc.cfg: /home/bernd/Programme/Laz-ESP8266/fpc/bin/x86_64-linux)

searchpath for fppkg user-specific packages
-Fu/home/bernd/Programme/Laz-ESP8266/packages.fppkg/units/$fpctarget/*
-Fu/home/bernd/Programme/Laz-ESP8266/cross/units/*

Nun können die Bindings in uses eingebunden werden und werden gefunden!
Falls Probleme mit der Verknüpfung auftreten unter Werkzeuge :



Für die inc Dateien (zum Bsp.: freertosconfig.inc) muss der Pfad in der IDE gesetzt werden.

Projekt, Prokekteinstellungen, Include-Dateien, ganz rechts auf die … klicken und Pfad setzen.
Tipp: lässt sich auch als Vorlage abspeichern, einfach Haken setzen.

Der Fehler 203:

Runtime error 203 is thown when the memory manager runs out of heap space (by default no memory is allocated to the heap of the default RTL memory manager). This can be fixed by either using fmem (which uses the freertos memory manager) or by the -Ch option (which allocates heap memory for the default RTL memory manager).

Der Laufzeitfehler 203 wird ausgelöst, wenn dem Speichermanager der Heap-Speicherplatz ausgeht (standardmäßig wird dem Heap des standardmäßigen RTL-Speichermanagers kein Speicher zugewiesen). Dies kann entweder durch die Verwendung von fmem (das den Freertos-Speichermanager verwendet) oder durch die Option -Ch (die Heap-Speicher für den standardmäßigen RTL-Speichermanager zuweist) behoben werden.

Quelle:
https://forum.lazarus.freepascal.org/index.php/topic,72082.msg563975.html#msg563975

Projekt 03_Uart_01

Das erste Programm mit Baudrate setzen und Ausgabe auf die serielle Schnittstelle:

```
program project1;
uses uart, esp_err;

procedure vTaskDelay(xTicksToDelay: uint32); external;

const
 UART_PORT: Tuart_port = UART_NUM_0;
 msg = 'UART test message'#13#10;

var
  uart_cfg: Tuart_config;

begin
 //mit den Libraries von fpcupdeluxe (9600*40) div 26 mit ccrause einfach 9600!
 uart_cfg.baud_rate  := 9600;//(9600*40) div 26;
 //uart_cfg.baud_rate  := (74880*40) div 26;
 //uart_cfg.baud_rate  := (115200*40) div 26;

 uart_cfg.data_bits  := UART_DATA_8_BITS;
 uart_cfg.parity     := UART_PARITY_DISABLE;
 uart_cfg.stop_bits  := UART_STOP_BITS_1;
 uart_cfg.flow_ctrl  := UART_HW_FLOWCTRL_DISABLE;
 EspErrorCheck(uart_param_config(UART_PORT, @uart_cfg));
 EspErrorCheck(uart_driver_install(UART_PORT, 256, 0, 0, nil, 0));
 repeat
  writeln('Ich bin eine Ausgabe mit writeln!');
  uart_write_bytes(UART_PORT, @msg[1], length(msg));
  vTaskDelay(300);
 until false;
end.
```

Ausgabe über CuteCom (Baudrate setzen nicht vergessen!):

```
09:08:47:888 Ich bin eine Ausgabe mit writeln!
09:08:47:925 UART test message
09:08:52:504 Ich bin eine Ausgabe mit writeln!
09:08:52:542 UART test message
09:08:57:120 Ich bin eine Ausgabe mit writeln!
09:08:57:158 UART test message
09:09:01:737 Ich bin eine Ausgabe mit writeln!
09:09:01:774 UART test message
```

Ausgabe über das Terminal:

Unter /home/bernd/Programme/Laz-ESP8266/cross/bin/xtensa-freertos/esp-rtos-3.4/tools findet man die Datei idf_monitor.py. Diese mit einem Texteditor öffnen.
In der ersten Zeile stand bei mir:
#!/usr/bin/env python
Das so ändern das die Python Version unter env verwendet wird:

#!/home/bernd/Programme/Laz-ESP8266/cross/bin/xtensa-freertos/esp-rtos-3.4/venv/bin/python3
(*Ich hab das gleich mit allen Dateien in dem Ordner gemacht ist aber glaube ich nicht nötig*)

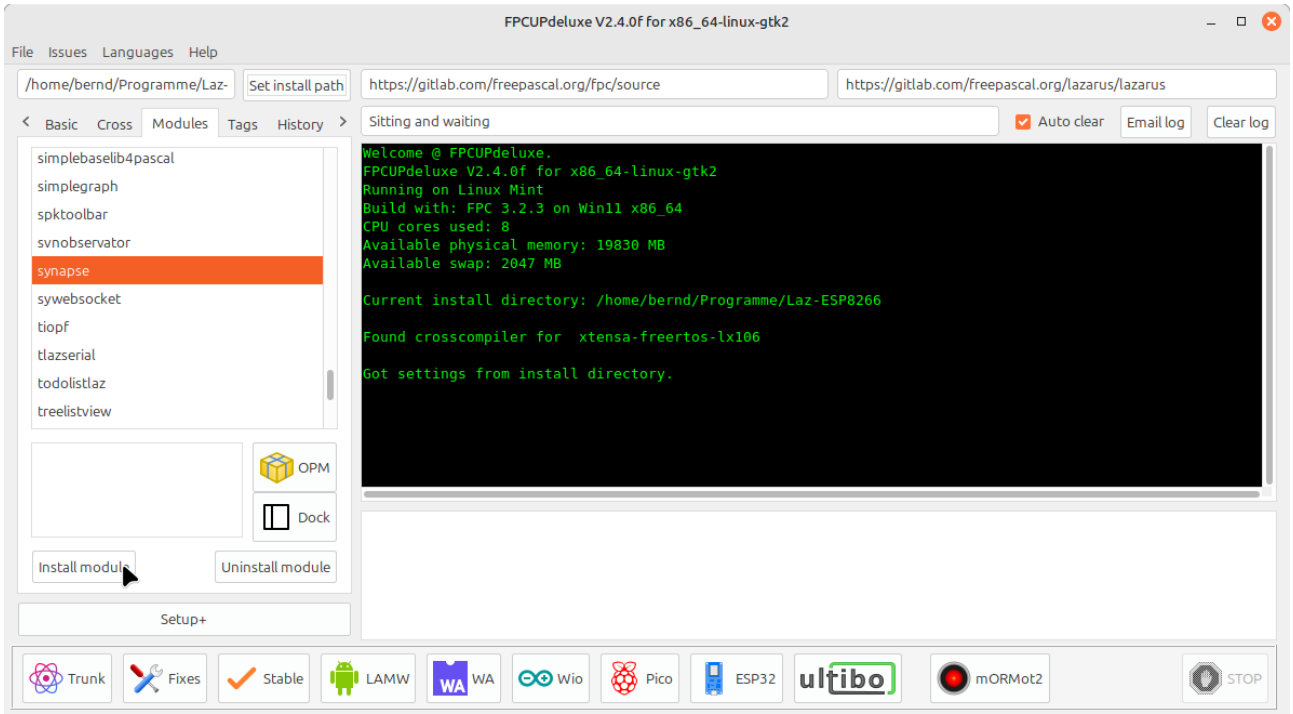
Terminal öffnen und folgendes eingeben:
Hinweis: Wird Baudrate 74880 verwendet sieht man den Boottext.

```
(source ~/.bash_profile
pyenv global 3.7.17)

cd /home/bernd/Programme/Laz-ESP8266/cross/bin/xtensa-freertos/esp-rtos-3.4/
tools
./idf_monitor.py --port /dev/ttyUSB0 --baud 74880
/home/bernd/Sprachen/Lazarus/64_bit/Meine_Packages/ESP8266/Demo_Projekte/
03_Uart_01/project1.elf
```

UART Monitor

Als nächstes kam ich auf die Idee einen einfachen Seriellen Monitor in meine Lazarus IDE zu integrieren.
Ich gab ihn den Namen UART Monitor.

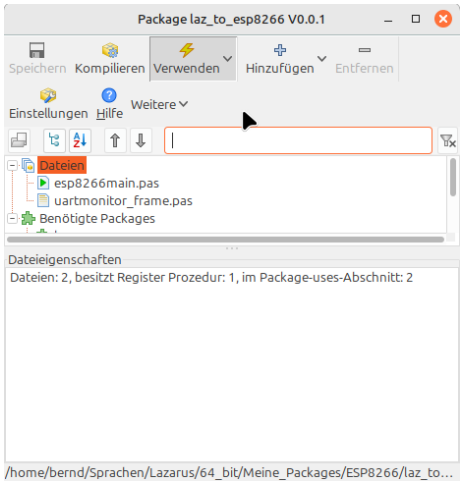
Der UART Monitor ist ein einfach gehaltener serieller Monitor zur Kommunikation mit dem ESP8266. **Er benötigt Synapse Synaser**! Bitte die dort enthaltenen Lizenzbedingungen durch lesen!
Synapse Synaser mit FpcUpDeluxe installieren. FpcUpDeluxe öffnen, kontrollieren ob das richtige Installverzeichnis gesetzt ist. Dann auf Modules klicken und synapse anwählen. Nun Install Module klicken und warten bis alles fertig installiert ist.



Damit Lazarus synapse findet muss man einmal in Lazarus zur laz_synapse.lpk navigieren und die doppelklicken. Pfad: /home/bernd/Programme/Laz-ESP8266/ccr/synapse

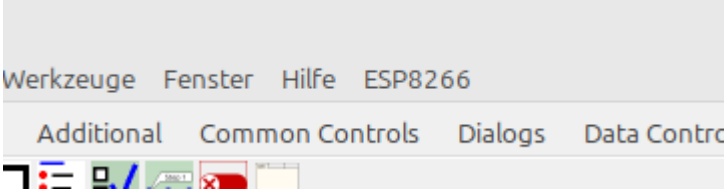
https://www.lazarusforum.de/viewtopic.php?p=88086#p88086

Der UART Monitor befindet sich im Package laz_to_esp8266.lpk.
Das Package laz_to_esp8266.lpk beeinhaltet momentan nur den UART Monitor, soll aber in Zukunft noch einige Tools zur einfacheren Programmierung des ESP8266 bekommen. Nach erfolgter Installation befindet sich im Mainmenü ein zusätzlicher Eintrag „ESP8266“.



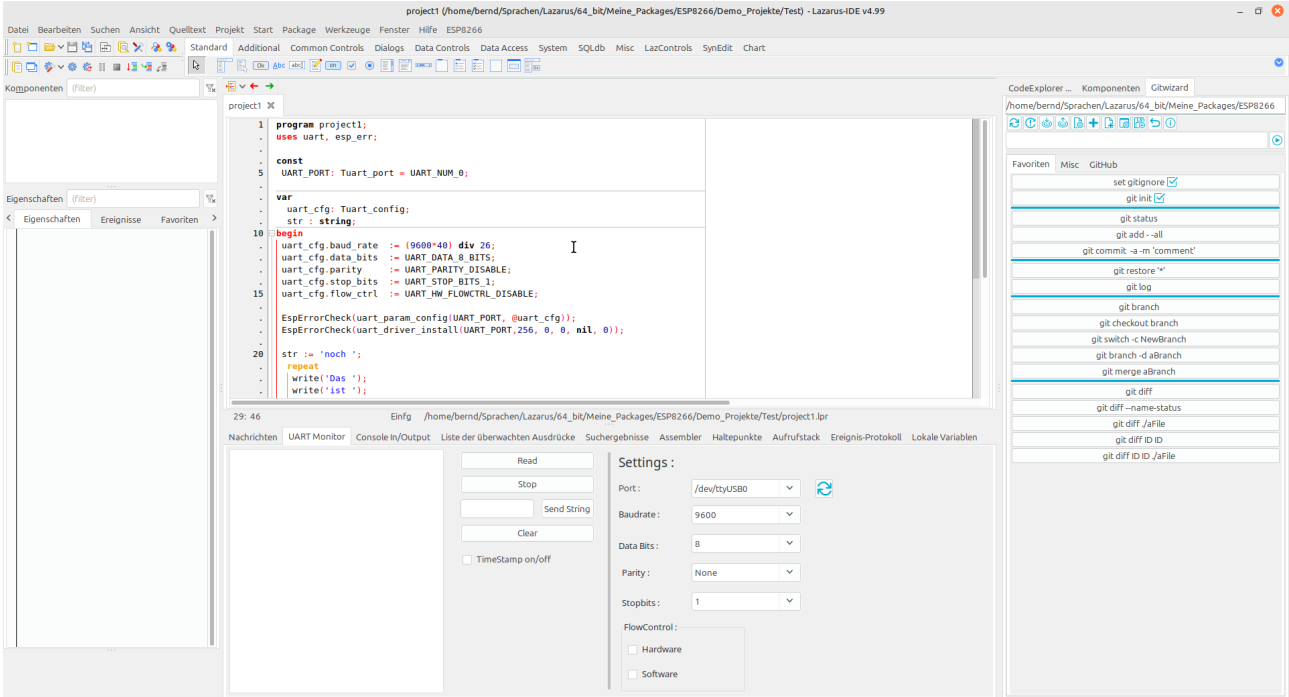
Um das Package zu installieren : Package, Package-Datei(*.lpk) öffnen. Zur laz_to_esp8266.lpk navigieren und doppelklicken.
Hier Verwenden und Installieren klicken.

Nach erfolgreicher Installation befindet sich ein neuer Eintrag „ESP8266“ im Hauptmenü.



Dort auf das Untermenü UART Monitor klicken.

Der UART Monitor.
Das Fenster ist in der IDE andockbar und es könnte zum Beispiel so aussehen:



Der UART Monitor benutzt die Funktion TBlockSerial.RecvString(Timeout: Integer): AnsiString; Damit lassen sich recht leicht gute Ergebnisse erzielen. Diese Funktion hat jedoch folgende Einschränkung:
This method waits until a terminated data string is received. The string is terminated by a CR/LF sequence. The resulting string is returned without the terminator (CR/LF)!

Das bedeutet schreibt man ein Programm in dem ausschließlich write zum Ausgeben benutzt wird friert das Programm ein bis der Watchdog Timer auslöst. Sobald ein writeln (oder LineEnding) zwischen repeat und until dabei ist gibt es keine Probleme.

Zum Senden verwende ich die Prozedur sendstring. Es wird von Synaser kein Terminator angehängt. Ich versende im UART Monitor noch ein chr(0) nach dem String um eine bessere Auswertbarkeit beim Empfangen zu haben. Wer das nicht möchte kann in der Unit „uartmonitor_frame“ in der Prozedure „Button_SendStringClick“ die chr(0) wegnehmen.

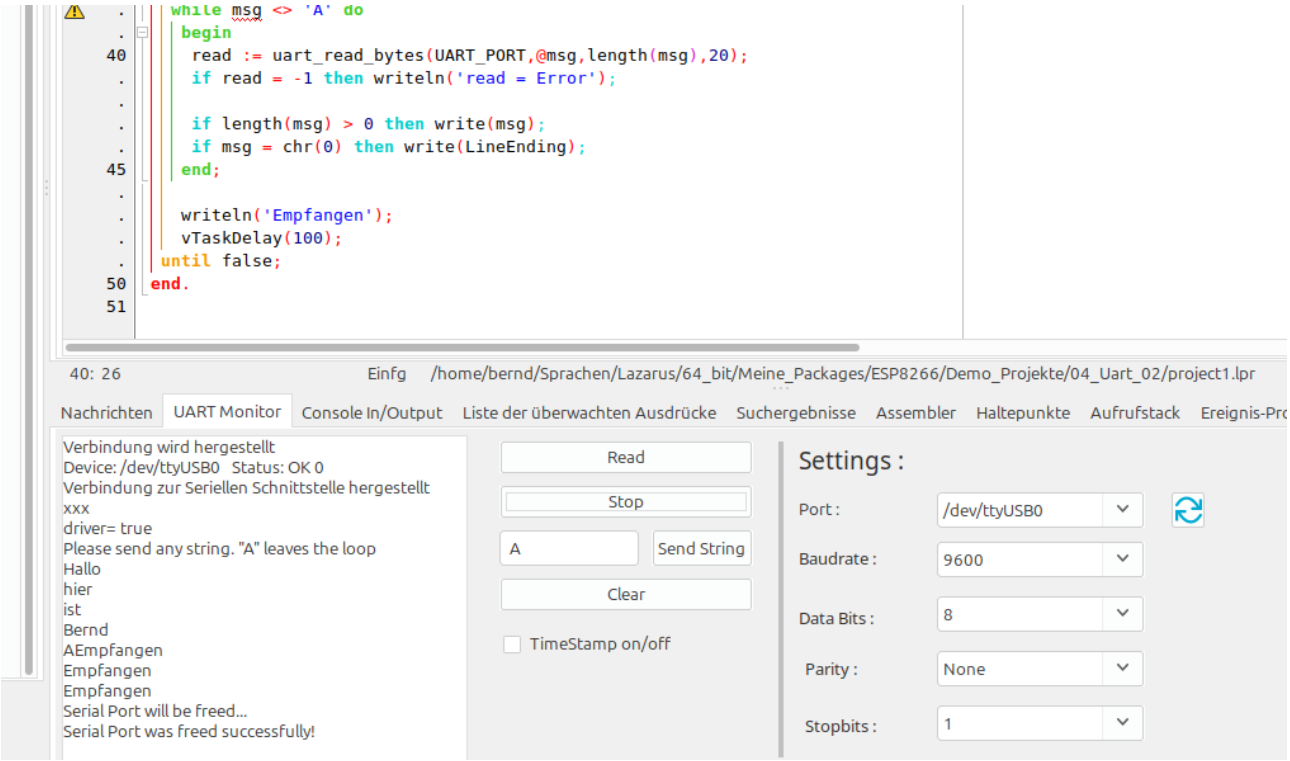
Synaser (bzw. Linux) unterstützt nicht die Baudrate von 74880!!!!

Programm 04_Uart_02

Auf eine Eingabe reagieren.

```
program project1;
uses uart, esp_err;

procedure vTaskDelay(xTicksToDelay: uint32); external;

const
 UART_PORT: Tuart_port = UART_NUM_0;

var
  uart_cfg: Tuart_config;
  config  : boolean;
  driver  : boolean;
  msg     : Char;
  read    : longint;

begin
 uart_cfg.baud_rate  := 9600;//(9600*40) div 26;
 uart_cfg.data_bits  := UART_DATA_8_BITS;
 uart_cfg.parity     := UART_PARITY_DISABLE;
 uart_cfg.stop_bits  := UART_STOP_BITS_1;
 uart_cfg.flow_ctrl  := UART_HW_FLOWCTRL_DISABLE;

 driver := false;
 config := false;

 config := EspErrorCheck(uart_param_config(UART_PORT, @uart_cfg));
//liefert true wenn okay
 driver := EspErrorCheck(uart_driver_install(UART_PORT, 256, 0, 0, nil, 0));
//liefert true wenn okay
 vTaskDelay(1000);
 if config then writeln('config= true') else writeln('config= false');
 if driver then writeln('driver= true') else writeln('driver= false');
 writeln('Please send any string. "A" leaves the loop');

 repeat
  while msg <> 'A' do
   begin
    read := uart_read_bytes(UART_PORT,@msg,length(msg),20);
    if read = -1 then writeln('read = Error');

    if length(msg) > 0 then write(msg);
    if msg = chr(0) then write(LineEnding);
   end;

   writeln('Empfangen');
   vTaskDelay(100);
 until false;
end.
```

Bei dem Programm habe ich meinen UART Monitor benutzt. Beliebigen Text eingeben und senden. Sobald man ein A sendet wird die Schleife verlassen.



Programm 05_Uart_03 testet die Ausgabe mittels Stringvariable und Constantstring.

Im Programm 06_Operators teste ich verschiedene Operatoren und Zuweisungen auf Funktion. Vorsicht nicht alles was sich problemlos kompilieren lässt läuft auch. Fazit man sollte die Unit Sysutils meiden und alles mit der Unit system zusammen bauen.

https://www.freepascal.org/docs-html/rtl/system/index.html

Unit Laz_ESP

In der Unit Laz_ESP möchte ich diverse Methoden sammeln um das Programmieren des ESP8266 etwas zu vereinfachen. (Edit: bitte nicht zu kritisch betrachten, bin noch am lernen)
Speicherort: /home/bernd/Programme/Laz-ESP8266/cross/units/additionally
Als erstes habe ich dort eine Funktion SerialBegin hinterlegt.

```
function SerialBegin(aBaudrate: longint) : boolean;
var bo : boolean;
begin
 Result := true;
 uart_cfg.baud_rate  := aBaudrate;//(aBaudrate*40) div 26;
 uart_cfg.data_bits  := UART_DATA_8_BITS;
 uart_cfg.parity     := UART_PARITY_DISABLE;
 uart_cfg.stop_bits  := UART_STOP_BITS_1;
 uart_cfg.flow_ctrl  := UART_HW_FLOWCTRL_DISABLE;
 bo := EspErrorCheck(uart_param_config(UART_PORT, @uart_cfg)); //liefert true wenn okay
 if not bo then Result := false;
 bo := EspErrorCheck(uart_driver_install(UART_PORT, 256, 0, 0, nil, 0)); //liefert true wenn okay
 if Result then if not bo then Result := false;
end;
```

Des weiteren Sleep:

```
procedure sleep(Milliseconds: cardinal);
begin
  vTaskDelay(Milliseconds div portTICK_PERIOD_MS);
end;
```

procedure vTaskDelay(xTicksToDelay: TTickType); external;
vTaskDelay(Ticks) – blockiert den aktuellen Task für eine bestimmte Anzahl von Ticks .
In der SDKConfig bzw. freertosconfig.inc findet man:
configTICK_RATE_HZ = 100
In der Unit portmacro ist portTick_Period_MS deklariert:
portTICK_PERIOD_MS = 1000 div configTICK_RATE_HZ;
Das bedeutet Tickdauer: 1000 div 100 = 10; 1 Tick = 10 ms.
Gibt man also 500 ein: 500 div 10 = 50 Es wird 50 Ticks gewartet!

pdMS_TO_TICKS:
Man kann auch vTaskDelay(pdMS_TO_TICKS(DelayMS)); verwenden muss dann aber noch die unit projdefs einbinden. Dort findet man:

```
function pdMS_TO_TICKS(xTimeInMs: TTickType): uint32;
begin
   pdMS_TO_TICKS := xTimeInMs * configTICK_RATE_HZ div 1000;
end;
```

procedure vTaskDelayUntil(pxPreviousWakeTime: PTickType; xTimeIncrement: TTickType); external;
Task wird genau alle TimeIncrement Ticks geweckt, unabhängig davon, wie lange die Ausführung dauert.

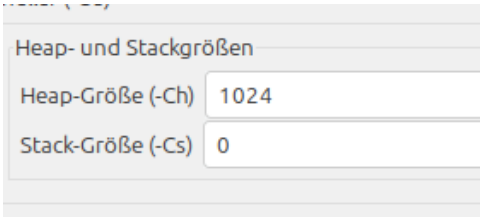
```
Uses portmacro, task …
xLastWakeTime: TTickType;
xLastWakeTime := xTaskGetTickCount(); // aktuelle Zeit
vTaskDelayUntil(@xLastWakeTime, 1000 div portTICK_PERIOD_MS);
```

Das Programm 07_Methoden ist zum Testen dieser Unit da. Außerdem ist dort die Unit sysutils eingebunden um zu testen was von dort funktioniert. Vieles lässt sich zwar kompilieren erzeugt aber

einen Laufzeitfehler 203. Mein Fazit ist die Unit sysutils besser nicht zu verwenden sondern auf die Möglichkeiten in der unit system zurück zugreifen.
https://www.freepascal.org/docs-html/rtl/system/index.html

Der Laufzeitfehler 203 wird ausgelöst, wenn dem Speichermanager der Heap-Speicherplatz ausgeht (standardmäßig wird dem Heap des standardmäßigen RTL-Speichermanagers kein Speicher zugewiesen). Dies kann entweder durch die Verwendung von fmem (das den Freertos-Speichermanager verwendet) oder durch die Option -Ch (die Heap-Speicher für den standardmäßigen RTL-Speichermanager zuweist) behoben werden.

Ich verwende die Unit fmem. Wer lieber selber Speicher zuweisen möchte kann das bei Projekt, Projekteinstellungen, Kompilieren und Linken, Heap-Größe (-CH) eintragen.



Quelle:
https://forum.lazarus.freepascal.org/index.php/topic,72082.msg563975.html#msg563975

Übringens wenn am Ende der Ausgabe Exit Code 0 erscheint ist das Programm ohne Fehler durch gelaufen.

Das Programm 08_WifiScan

```
//Das Programm stammt ursprünglich von hier:
//https://github.com/ccrause/fpc-esp-freertos/tree/master/examples/wifiscan
//Nur leicht von mir verändert.

program wifiscan;

{$include sdkconfig.inc}

uses
  esp_err, esp_wifi, esp_wifi_types, laz_esp, nvs,
  nvs_flash, esp_event_legacy;


function eventhandler(ctx: pointer; event: Psystem_event): Tesp_err;
begin
  writeln('Event -  ', event^.event_id);
  Result := ESP_OK;

end;


procedure printAPInfo(const AP: Twifi_ap_record);
var
  pb: PByte;
begin
  pb := @AP.ssid[0];
  while (pb^ > 0) and (pb - @AP.ssid[0] < 33) do
  begin
    if pb^ > 31 then
      write(char(pb^))
    else
      write('.');
    inc(pb);
  end;
  writeln;

  write('  bssid: ');
  pb := @AP.bssid[0];
  while (pb^ > 0) and (pb - @AP.bssid[0] < 6) do
  begin
    write(HexStr(pb^,2), ' ');
    inc(pb);
  end;
  writeln;

  writeln('  primary: ', AP.primary);
  if ord(AP.second) <= ord(WIFI_SECOND_CHAN_BELOW) then
    writeln('  secondary: ', AP.second)
  else
    writeln('  secondary: INVALID');
  writeln('  rssi: ', AP.rssi);
  if ord(AP.authmode) < ord(WIFI_AUTH_MAX) then
    writeln('  AuthMode: ', AP.authmode)
  else
    writeln('  AuthMode: INVALID');
  if ord(AP.pairwise_cipher) <= ord(WIFI_CIPHER_TYPE_UNKNOWN) then
    writeln('  Pairwise cipher: ', AP.pairwise_cipher)
  else
    writeln('  Pairwise cipher: INVALID');
  if ord(AP.group_cipher) <= ord(WIFI_CIPHER_TYPE_UNKNOWN) then
    writeln('  Group cipher: ', AP.group_cipher)
  else
    writeln('  Group cipher: INVALID');
  if ord(AP.ant) <= ord(WIFI_ANT_MAX) then
    writeln('  Antenna: ', AP.ant)
  else
    writeln('  Antenna: INVALID');
  writeln('  Supported wifi modes');
  writeln('    b: ', boolean(AP.phy_11b));
  writeln('    g: ', boolean(AP.phy_11g));
  writeln('    n: ', boolean(AP.phy_11n));
```

```
  writeln('    lr: ', boolean(AP.phy_lr));
  with AP.country do
  begin
    writeln('  Country info');
    write('    Country code: ');
    pb := @AP.country.cc[0];
    while (pb^ > 0) and (pb - @AP.country.cc[0] < 3) do
    begin
      if pb^ > 31 then
        write(char(pb^))
      else
        write('.');
      inc(pb);
    end;
    writeln;

    writeln('    Start channel: ', schan);
    writeln('    End channel: ', nchan);
    writeln('    Max power: ', max_tx_power);
    if ord(policy) <= ord(WIFI_COUNTRY_POLICY_MANUAL) then
      writeln('    Policy: ', policy)
    else
      writeln('    Policy: INVALID');
  end;
end;

procedure printAP_SSID(const AP: Twifi_ap_record);
var pb: PByte;
begin
  pb := @AP.ssid[0];
  while (pb^ > 0) and (pb - @AP.ssid[0] < 33) do
  begin
    if pb^ > 31 then
      write(char(pb^))
    else
      write('.');
    inc(pb);
  end;
  writeln;
end;

procedure wifi_scan;
const
  number = 10;
var
  cfg        : Twifi_init_config;
  ap_info    : array[0..number-1] of Twifi_ap_record;
  ap_info_len: uint16 = number;
  i          : integer;
begin
  writeln('Enter scan');
  EspErrorCheck(esp_event_loop_init(@eventhandler, nil));
  WIFI_INIT_CONFIG_DEFAULT(cfg);
  EspErrorCheck(esp_wifi_init(@cfg));
  EspErrorCheck(esp_wifi_set_mode(WIFI_MODE_STA));
  EspErrorCheck(esp_wifi_start());


  repeat
   EspErrorCheck(esp_wifi_scan_start(nil, true));
   ap_info_len := length(ap_info);
   EspErrorCheck(esp_wifi_scan_get_ap_records(@ap_info_len, @ap_info[0]));
   EspErrorCheck(esp_wifi_scan_get_ap_num(@ap_info_len));

   if ap_info_len > 0 then
     for i := 0 to ap_info_len-1 do
      begin
       write('AP #', i, ': ');
       printAPInfo(ap_info[i]);
       writeln;
      end
   else
    writeln('No AP records returned.');

   sleep(1000);

   EspErrorCheck(esp_wifi_scan_start(nil, true));
```

```
    ap_info_len := length(ap_info);
    EspErrorCheck(esp_wifi_scan_get_ap_records(@ap_info_len, @ap_info[0]));
    EspErrorCheck(esp_wifi_scan_get_ap_num(@ap_info_len));
    writeln('Liste: ');
    if ap_info_len > 0 then
      for i := 0 to ap_info_len-1 do
       printAP_SSID(ap_info[i]);
    writeln;
  until false;
end;

var
  ret: longint;

begin
  serialbegin(9600);

  ret := nvs_flash_init();
  if (ret = ESP_ERR_NVS_NO_FREE_PAGES) then
  begin
    nvs_flash_erase();
    ret := nvs_flash_init();
  end;
  wifi_scan();
end.
```

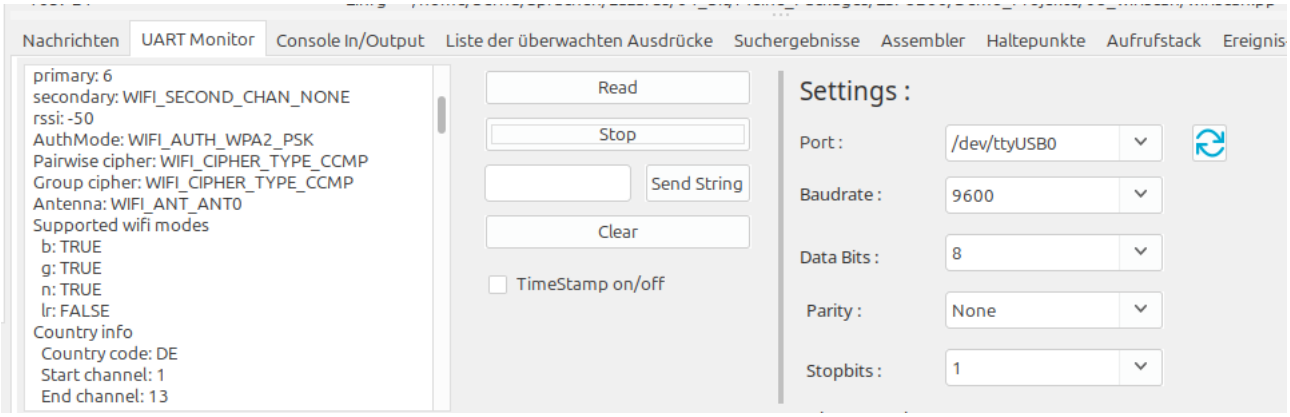
Damit das Programm lief musste ich wie weiter oben beschrieben die Libraries tauschen!

The fpcupdeluxe installed esp8266-rtos-sdk libraries do not appear to support the wifiscan functionality. To update the esp8266 libraries, extract the libs folder from the esp8266 snaptshot and copy the libraries (libxxx.a) into the fpcupdeluxe libraries folder (fpcupdeluxefolder/cross/lib/xtensa-freertos/lx106/), overwriting the old libraries.

Die von fpcupdeluxe installierten esp8266-rtos-sdk-Bibliotheken scheinen die Wifiscan-Funktionalität nicht zu unterstützen. Um die esp8266-Bibliotheken zu aktualisieren, extrahieren Sie den Ordner „libs" aus dem esp8266-Snapshot und kopieren Sie die Bibliotheken (libxxx.a) in den Bibliotheksordner von fpcupdeluxe (fpcupdeluxefolder/cross/lib/xtensa-freertos/lx106/), wobei Sie die alten Bibliotheken überschreiben.

Quelle:
https://forum.lazarus.freepascal.org/index.php/topic,72082.msg563683.html#msg563683

Die Serielle Ausgabe liefert eine Liste der erkannten Wlan Netze und Details darüber.



Das Programm 09_SimpleHttpServer

```
//Das Programm stammt von hier:
//https://github.com/ccrause/fpc-esp-freertos/tree/master/examples/simplehttpserver
//Ich habe wificonnect durch wificonnect2 ersetzt
//zusätzlich fmem eingebunden, sont runtime error 203
//meine unit laz_esp eingebunden und eine Baudrate von 9600 gesetzt
//AP_Name und PWD direkt als Konstante hinterlegt, muss noch eingetragen werden!!!!

program simplehttpserver;

uses
  fmem,
  wificonnect2,
  esp_http_server, esp_err, http_parser,
  portable, task, laz_esp;

{$macro on}
{$inline on}

const
  htmlpage = '<!DOCTYPE html><html><body><h1>Welcome to FPC + ESP-IDF</h1><p><img src="/fpclogo.gif"
alt="FPC"></p></body></html>';
  AP_NAME  = 'DEINE_SSID';
  PWD      = 'DEIN_Netzwerkschlüssel';

// Import fpc logo in gif format as array of char
{$include fpclogo.inc}

function hello_get_handler(req: Phttpd_req): Tesp_err;
var
  buf: PChar;
  buf_len: uint32;
begin
  buf_len := httpd_req_get_hdr_value_len(req, 'Host') + 1;
  if (buf_len > 1) then
  begin
    buf := pvPortMalloc(buf_len);
    if (httpd_req_get_hdr_value_str(req, 'Host', buf, buf_len) = ESP_OK) then
      writeln('Found header => Host: ', buf);
    vPortFree(buf);
  end;

  httpd_resp_send(req, htmlpage, length(htmlpage));

  result := ESP_OK;
end;

function fpclogo_get_handler(req: Phttpd_req): Tesp_err;
var
  buf: PChar;
  buf_len: uint32;
begin
  buf_len := httpd_req_get_hdr_value_len(req, 'Host') + 1;
  writeln('buff_len ',buf_len);
  if (buf_len > 1) then
  begin
    buf := pvPortMalloc(buf_len);
    if (httpd_req_get_hdr_value_str(req, 'Host', buf, buf_len) = ESP_OK) then
      writeln('Found header => Host: ', buf);
    writeln('nach:',buf);
    vPortFree(buf);
  end;

  httpd_resp_set_type(req, 'image/gif');
  httpd_resp_send(req, fpclogo, length(fpclogo));
  result := ESP_OK;
end;



function start_webserver: Thttpd_handle;
var
  server: Thttpd_handle;
```

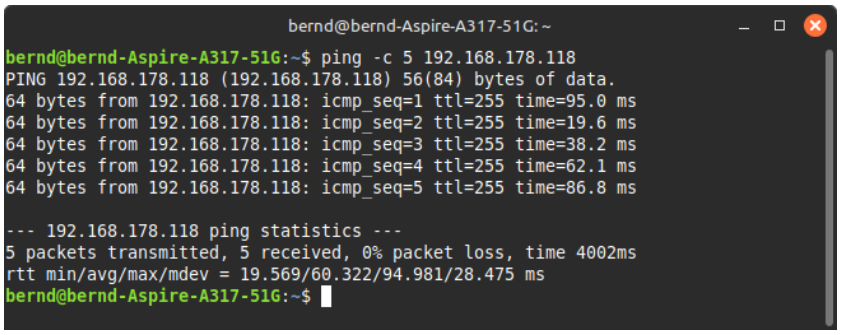
```
  config: Thttpd_config;
  helloUriHandlerConfig: Thttpd_uri;
  fpcUriHandlerConfig: Thttpd_uri;
begin
  config := HTTPD_DEFAULT_CONFIG();

  with helloUriHandlerConfig do
  begin
    uri       := '/';
    method    := HTTP_GET;
    handler   := @hello_get_handler;
    user_ctx  := nil;
  end;

  with fpcUriHandlerConfig do
  begin
    uri       := '/fpclogo.gif';
    method    := HTTP_GET;
    handler   := @fpclogo_get_handler;
    user_ctx  := nil;
  end;

  writeln('Starting server on port: ', config.server_port);
  if (httpd_start(@server, @config) = ESP_OK) then
  begin
    // Set URI handlers
    writeln('Registering URI handler');
    httpd_register_uri_handler(server, @helloUriHandlerConfig);
    httpd_register_uri_handler(server, @fpcUriHandlerConfig);
   result := server;
  end
  else
  begin
    result := nil;
    writeln('### Failed to start httpd');
  end;
end;

begin
  SerialBegin(9600);

  connectWifiAP(AP_NAME, PWD);

  writeln('Starting web server...');
  start_webserver;

  repeat
    vTaskDelay(10);
  until false;
end.
```

Siehe auch:
https://forum.lazarus.freepascal.org/index.php/topic,72082.msg563951.html#msg563951

Die Ausgabe im Seriellen Monitor sollte in etwa so aussehen:

```
<0x1b>[0;32mI (605) wifi:state: 0 -> 2 (b0)

<0x1b>[0m<0x1b>[0;32mI (614) wifi:state: 2 -> 3 (0)
<0x1b>[0m<0x1b>[0;32mI (622) wifi:state: 3 -> 5 (10)
<0x1b>[0m<0x1b>[0;32mI (656) wifi:connected with FRITZ!Box, aid = 9, channel 6, HT20, bssid =
44:4f:6d:1d:f4:d8
<0x1b>[0mReceived Wifi event: WIFI_EVENT_STA_CONNECTED
<0x1b>[0;32mI (1703) tcpip_adapter: sta ip: 192.168.178.118, mask: 255.255.255.0, gw:
192.168.178.1<0x1b>[0m
Got ip: 192.168.178.118
Connected. Test connection by pinging the above IP address from the same network
Starting web server...
Starting server on port: 80
Registering URI handler
```

Gibt man im Terminal `ping -c 5 192.168.178.118` ein sollte es so aussehen:



Gibt man die IP im Browser ein sollte das zu sehen sein:

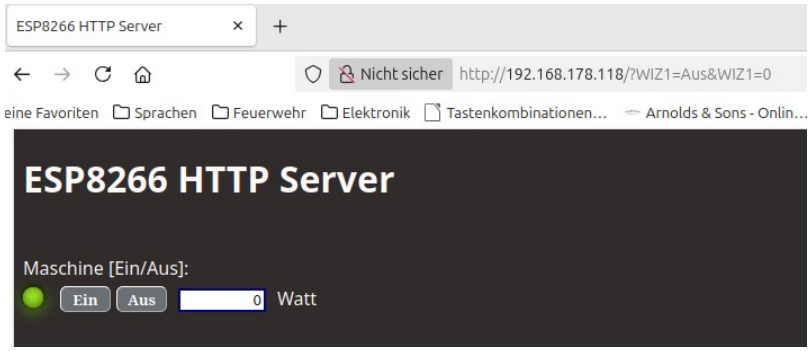
MyFirstHttpServer

Im Projekt MyFirstHttpServer möchte ich einen minimalen Webserver programmieren mit dem sich eine Funksteckdose mit integriertem Stromverbrauchsmesser auslesen und schalten lässt.

Im Programm 10_MyFirstHttpServer_01 habe ich das Programm 09_SimpleHttpServer so abgeändert das der HTML Text in einer eigenen Prozedur steht und erst mal alles was nicht nötig ist weggelassen wurde.

Ausgabe im Browser:

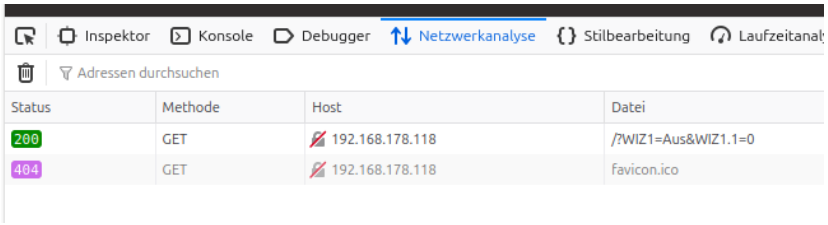
Im Programm 11_MyFirstHttpServer_02 hab ich den HTML Text vervollständigt. Zwei Submit-Buttons und ein Label eingefügt.



In der Funktion main_get_handler hab ich folgenden Code eingefügt:

```
buf_len := httpd_req_get_url_query_len(req) +1;
  if buf_len > 1 then
   begin
    buf := pvPortMalloc(buf_len);
    if httpd_req_get_url_query_str(req, buf,buf_len) = ESP_OK then
     writeln('Query: ', buf);
    vPortFree(buf);
   end;
```

Hintergrund:
Ein Klick auf einen Submit-Button löst einen Get Request aus. Dies kann man sich im Browser anzeigen lassen. Rechtsklick, Untersuchen, Netzwerkanalyse (Firefox).



Quelle:
https://www.ionos.de/digitalguide/hosting/hosting-technik/http-request-erklaert/

*Der GET-Anfrage können **zusätzliche Informationen** mitgegeben werden, die der Webserver verarbeiten soll. Diese URL-Parameter werden einfach an die URL angehängt. Die Syntax ist ganz einfach:*

- *Der Query-String wird mit einem „?“ (Fragezeichen) eingeleitet.*
- *Jeder Parameter ist benannt, besteht also aus einem Namen und einem Wert: „Name=Wert“*
- *Wenn mehrere Parameter mitgegeben werden sollen, dann werden sie mit einem „&“ verbunden.*

Im Seriellen Monitor sieht das dann so aus:

```
GetHandler
Found header => Host: 192.168.178.118
Query: WIZ1=Aus&WIZ1.1=0
CreateWebpage.....
```

Wobei hier der Aus-Button gedrückt wurde (WIZ1=Aus). Der Wert des Labels wird angehängt (WIZ1.1).

Im Programm 12_MyFirstHttpServer_03 habe ich die Unit sysutils eingefügt um die Funktion Pos verwenden zu können. Dann habe ich eine Abfrage in den main_get_handler eingefügt welcher Button gedrückt wurde.

```
i := Pos('WIZ1=Ein',buf);
    if i <> 0 then
     begin
      Wiz1 := true;
      writeln('WIZ1 ist ein');
     end;
i := Pos('WIZ1=Aus',buf);
   if i <> 0 then
     begin
      Wiz1 := false;
      writeln('WIZ1 ist aus');
     end;
```

Um in den HTML Text eine Abfrage zu integrieren habe ich den HTML Text in mehrere Teile zerstückelt:

```
          '</head>'+
          '<form>'+
            '<h1><font color="snow">ESP8266 HTTP Server</font></h1>'+
            '<br><font color="snow">Maschine [Ein/Aus]:</font></br>';

  if WIZ1 then Chunk2 := '<div class="led-green"></div>';

  if not WIZ1 then Chunk2 := '<div class="led-gray"></div>';

  Chunk3 :=  '<style>input{color:white}</style>'+
             '<input type="submit" name="WIZ1" value="Ein" style=''Backgro
             'font-family:Bradley Hand ITC; font-size: 24; font-weight: bo
```

Mit der function httpd_resp_send_chunk(r: Phttpd_req; buf: PChar;
  buf_len: Tsize): Tesp_err; external; lassen sich dann mehrere Chunks (=Brocken, Stücke) nacheinander übertragen.
Abgeschlossen wird mit nil!

```
  CreateWebpage;
  //httpd_resp_send(req, content, length(content));
  httpd_resp_send_chunk(req, Chunk1, length(Chunk1));
  httpd_resp_send_chunk(req, Chunk2, length(Chunk2));
  httpd_resp_send_chunk(req, Chunk3, length(Chunk3));
  // Signal finish of chunks
  httpd_resp_send_chunk(req, nil, 0);
  result := ESP_OK;
end;
```

Nun kann mit den Buttons „Aus“ und „Ein“ die LED zwischen Grün und Grau geschalten werden.
Im nächsten Schritt möchte ich nun die WIZ Smartplug (Funk-Steckdose) mittels UDP ansprechen.

UDP

Um den ESP8266 mit der WIZ Steckdose zu verbinden benötige ich Udp. Leider habe ich zum Thema UDP im Repository von ccrause keine Bindings gefunden.
Mein erster Anhaltspunkt war deshalb ein Beispiel im RTOS_SDK.
https://github.com/espressif/ESP8266_RTOS_SDK/blob/master/examples/protocols/sockets/udp_client/main/udp_client.c

Ich habe dieses Beispiel und die neu benötigten Bindings nach Pascal übersetzt. Leider sind meine C bzw. C++ Kenntnisse nicht so gut, so das ich gestehen muss etliche Hilfswerkzeuge in Anspruch genommen zu haben:
- H2PAS
- ChatGPT
- Copilot
- Online C to Pascal Converter (https://www.codeconvert.ai/c-to-pascal-converter)

Die passenden C-Header Dateien sind im Release 3.4 enthalten.
Link zum Release 3.4:
https://github.com/espressif/ESP8266_RTOS_SDK/releases
Download (mit *.h Dateien):
https://github.com/espressif/ESP8266_RTOS_SDK/releases/download/v3.4/ESP8266_RTOS_SDK-v3.4.zip

Alle neu erstellten Bindings habe ich in den Ordner /home/.../Laz-ESP8266/cross/units/additionally kopiert.

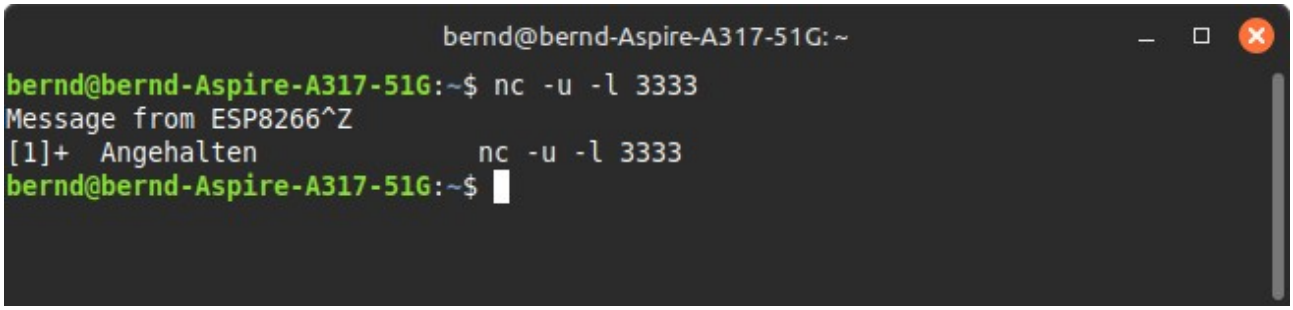
Das Programm 14_UDP_Senden_01

Das Programm lässt den ESP8266 den Text der in Payload gesetzt wurde an ein Gerät im Heimnetz senden.

```
payload : PChar = 'Message from ESP8266';

HOST_IP_ADDR = '192.168.178.52';  //Dies ist die IP meines Laptops im Heimnetz
```

Zum Prüfen des Programms kann netcat verwendet werden. Dies ist unter Mint bereits vorinstalliert.

Das Programm 14 auf den ESP übertragen (IP des Laptops ins Programm eingeben). ESP abstecken. Dann Terminal öffnen und `nc -u -l 3333` eingeben. Enter drücken!
ESP anstecken!



Mit STRG+Z kann angehalten werden.

```
program udp_client_example;

{$mode objfpc}{$H+}

uses
  fmem,
  SysUtils,
  lwip_sockets,
  task,
  esp_log2,
  portmacro,
  lwip_inet,
  lwip_def,
  wificonnect2;

const
  TAG = 'example';
  HOST_IP_ADDR = //'192.168.178.52'; // anpassen, das ist die IP des Gerätes an das gesendet wird
  PORT = 3333;

  payload : PChar = 'Message from ESP8266';

  AP_NAME  = //'DEINE_SSID';
  PWD      = //'DEIN_Netzwerkschlüssel';

procedure udp_client_task(pvParameters: Pointer);
var
  rx_buffer : array[0..127] of char;
  addr_str  : array[0..127] of char;
  addr_family : Integer;
  ip_protocol : Integer;

  destAddr : sockaddr_in;
  sourceAddr : sockaddr_in;
  sock : Integer;
  err : Integer;
  len : Integer;
```

```
  socklen : socklen_t;

begin
  while True do
  begin
    FillChar(destAddr, SizeOf(destAddr), 0);

    destAddr.sin_addr.s_addr := inet_addr(HOST_IP_ADDR);
    destAddr.sin_family := AF_INET;
    destAddr.sin_port := lwip_htons(PORT);

    addr_family := AF_INET;
    ip_protocol := IPPROTO_IP;

    inet_ntoa_r(destAddr.sin_addr, @addr_str, SizeOf(addr_str)-1);

    sock := lwip_socket(addr_family, SOCK_DGRAM, ip_protocol);

    if sock < 0 then
    begin
      ESP_LOGE(TAG, 'Unable to create socket',[err]);
      break;
    end;

    ESP_LOGI(TAG, 'Socket created',[err]);

    while True do
    begin
      err := lwip_sendto(
        sock,
        payload,
        strlen(payload),
        0,
        @destAddr,
        SizeOf(destAddr)
      );

      if err < 0 then
      begin
        ESP_LOGE(TAG, 'Error occurred during sending',[Err]);
        break;
      end;

      ESP_LOGI(TAG, 'Message sent',[Err]);

      socklen := SizeOf(sourceAddr);

      len := lwip_recvfrom(
        sock,
        @rx_buffer,
        SizeOf(rx_buffer) - 1,
        0,
        @sourceAddr,
        @socklen
      );

      if len < 0 then
      begin
        ESP_LOGE(TAG, 'recvfrom failed',[Err]);
        break;
      end
      else
      begin
        rx_buffer[len] := #0;
        ESP_LOGI(TAG, '%s %d', [rx_buffer, Err]);
      end;

      vTaskDelay(2000 div portTICK_PERIOD_MS);
    end;

    if sock <> -1 then
    begin
      ESP_LOGE(TAG, 'Shutting down socket and restarting...',[Err]);
      lwip_shutdown(sock, 0);
      lwip_close(sock);
    end;
  end;
```

```
  vTaskDelete(nil);
end;

begin

  connectWifiAP(AP_NAME, PWD);

  xTaskCreate(
    @udp_client_task,
    PChar('udp_client'),
    4096,
    nil,
    5,
    nil
  );
end.
```

Info zu der Funktion **xTaskCreate**:

```
xTaskCreate(
    @udp_client_task,
    PChar('udp_client'),
    4096,
    nil,
    5,
    nil
  );
```

Damit wird ein neuer Task (Thread) angelegt.

Bedeutung der Parameter:

- @udp_client_task
  → Zeiger auf die Funktion, die der Task ausführen soll
- PChar('udp_client')
  → Name des Tasks (Debug-Zwecke)
- 4096
  → Stackgröße des Tasks (in Bytes oder Words, je nach Port)
- nil
  → Parameter, die an den Task übergeben werden
- 5
  → Priorität des Tasks (Höherer Wert = höhere Priorität)
- nil
  → Handle (Referenz) auf den Task (Wenn du später den Task steuern willst (stoppen, suspendieren), gibst du hier eine Variable an)

Das Programm 15_UDP_Senden_02

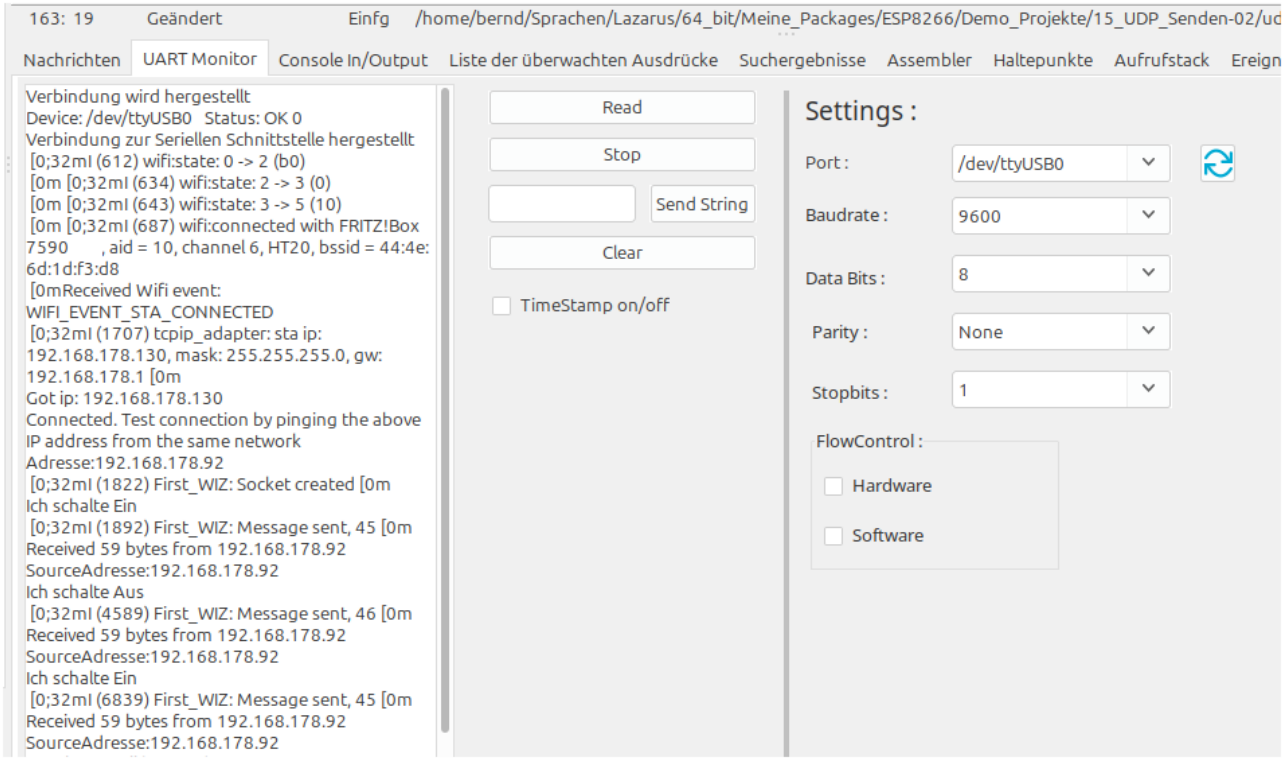
Im Programm 15 wird eine WIZ Smartplag (Funksteckdose) ca. alle 3 Sekunden ein und aus geschaltet. Die WIZ Geräte benötigen als Port 38899.
Der Einschaltbefehl lautet: {"method":"setState","params":{"state":true}}
Zum Ausschalten wird {"method":"setState","params":{"state":false}} gesendet.
https://seanmcnally.net/wiz-config.html

```
program udp_senden_WIZ;

{$mode objfpc}{$H+}

uses
  fmem,
  SysUtils,
  lwip_sockets,
  task,
  esp_log2,
  portmacro,
  lwip_inet,
  lwip_def,
  wificonnect2,
  Laz_ESP;

const
  TAG = 'First_WIZ';
  HOST_IP_ADDR = '192.168.178.92'; // anpassen, das ist die IP des Gerätes an
das gesendet wird
  PORT = 38899;   //Dieser Port muss für die WIZ Geräte verwendet werden!

  AP_NAME  = //'DEINE_SSID';
  PWD      = //'DEIN_Netzwerkschlüssel';

  const payload_on  : PChar = '{"method":"setState","params":{"state":true}}';
  const payload_off : PChar = '{"method":"setState","params":{"state":false}}';

var
  payload       : PChar;
  State         : boolean;

procedure udp_client_task(pvParameters: Pointer);
var
  rx_buffer : array[0..127] of char;
  addr_str  : array[0..127] of char;
  addr_family : Integer;
  ip_protocol : Integer;

  destAddr : sockaddr_in;
  sourceAddr : sockaddr_in;
  sock : Integer;
  err : Integer;
  len : Integer;
  socklen : socklen_t;

begin
  while True do
  begin
```

```
FillChar(destAddr, SizeOf(destAddr), 0);

destAddr.sin_addr.s_addr := inet_addr(HOST_IP_ADDR);
destAddr.sin_family := AF_INET;
destAddr.sin_port := lwip_htons(PORT);

addr_family := AF_INET;
ip_protocol := IPPROTO_IP;

inet_ntoa_r(destAddr.sin_addr, @addr_str, SizeOf(addr_str));
writeln('Adresse:',StrPas(@addr_str[0]));

sock := lwip_socket(addr_family, SOCK_DGRAM, ip_protocol);

if sock < 0 then
begin
  ESP_LOGE(TAG, 'Unable to create socket',[]);
  break;
end;

ESP_LOGI(TAG, 'Socket created',[]);



while True do
begin
  if not State then
   begin
    payload := payload_on;  //Einschalten
    State := true;
    writeln('Ich schalte Ein');
   end
  else
   begin
    payload := payload_off;  //Ausschalten
    State := false;
    Writeln('Ich schalte Aus');
   end;


  err := lwip_sendto(
    sock,
    payload,
    strlen(payload),
    0,
    @destAddr,
    SizeOf(destAddr)
  );

  if err < 0 then
  begin
    ESP_LOGE(TAG, 'Error occurred during sending, %d',[Err]);
    break;
  end;

  ESP_LOGI(TAG, 'Message sent, %d',[Err]);

  socklen := SizeOf(sourceAddr);

  len := lwip_recvfrom(
    sock,
```

```
        @rx_buffer,
        SizeOf(rx_buffer) -1,
        0,
        @sourceAddr,
        @socklen
      );

      if len < 0 then
      begin
        ESP_LOGE(TAG, 'recvfrom failed, %d',[Len]);
        break;
      end
      else
      begin
        rx_buffer[len] := #0;
        writeln('Received ',len,' bytes from ', StrPas(@addr_str[0]));
        ESP_LOGI(TAG, '%s , %d', [rx_buffer, Len]);

        inet_ntoa_r(sourceAddr.sin_addr, @addr_str, SizeOf(addr_str));
        writeln('SourceAdresse:',StrPas(@addr_str[0]));
      end;

      vTaskDelay(2000 div portTICK_PERIOD_MS);

    end;

    if sock <> -1 then
    begin
      ESP_LOGE(TAG, 'Shutting down socket and restarting... ,%d',[sock]);
      lwip_shutdown(sock, 0);
      lwip_close(sock);
    end;
  end;

  vTaskDelete(nil);
end;

begin
  SerialBegin(9600);

  connectWifiAP(AP_NAME, PWD);

  State := false;

  xTaskCreate(
    @udp_client_task,
    PChar('udp_client'),
    8192,//4096,
    nil,
    5,
    nil
  );

  while True do
  begin
   sleep(100);
  end;

end.
```

Ausgabe:


163: 19 Geändert Einfg /home/bernd/Sprachen/Lazarus/64_bit/Meine_Packages/ESP8266/Demo_Projekte/15_UDP_Senden-02/ud
Nachrichten
UART Monitor
Console In/Output
Liste der überwachten Ausdrücke
Suchergebnisse
Assembler
Haltepunkte
Aufrufstack
Ereign
Verbindung wird hergestellt
Device: /dev/ttyUSB0 Status: OK 0
Verbindung zur Seriellen Schnittstelle hergestellt
[0;32mI (612) wifi:state: 0 -> 2 (b0)
[0m [0;32mI (634) wifi:state: 2 -> 3 (0)
[0m [0;32mI (643) wifi:state: 3 -> 5 (10)
[0m [0;32mI (687) wifi:connected with FRITZ!Box 7590 , aid = 10, channel 6, HT20, bssid = 44:4e:6d:1d:f3:d8
[0mReceived Wifi event: WIFI_EVENT_STA_CONNECTED
[0;32mI (1707) tcpip_adapter: sta ip: 192.168.178.130, mask: 255.255.255.0, gw: 192.168.178.1 [0m
Got ip: 192.168.178.130
Connected. Test connection by pinging the above IP address from the same network
Adresse:192.168.178.92
[0;32mI (1822) First_WIZ: Socket created [0m
Ich schalte Ein
[0;32mI (1892) First_WIZ: Message sent, 45 [0m
Received 59 bytes from 192.168.178.92
SourceAdresse:192.168.178.92
Ich schalte Aus
[0;32mI (4589) First_WIZ: Message sent, 46 [0m
Received 59 bytes from 192.168.178.92
SourceAdresse:192.168.178.92
Ich schalte Ein
[0;32mI (6839) First_WIZ: Message sent, 45 [0m
Received 59 bytes from 192.168.178.92
SourceAdresse:192.168.178.92
Read
Stop
Send String
Clear
TimeStamp on/off
Settings :
Port :
/dev/ttyUSB0
Baudrate :
9600
Data Bits :
8
Parity :
None
Stopbits :
1
FlowControl :
Hardware
Software

Das Programm 16_UDP_Senden_03

Im Programm 16 wird aus einer WIZ Smartplug der aktuelle Stromverbrauch in Watt ausgelesen. Außerdem wird noch die IP der angesprochenen Funksteckdose zurück gegeben.
Befehl: `{"method": "getPower"}`

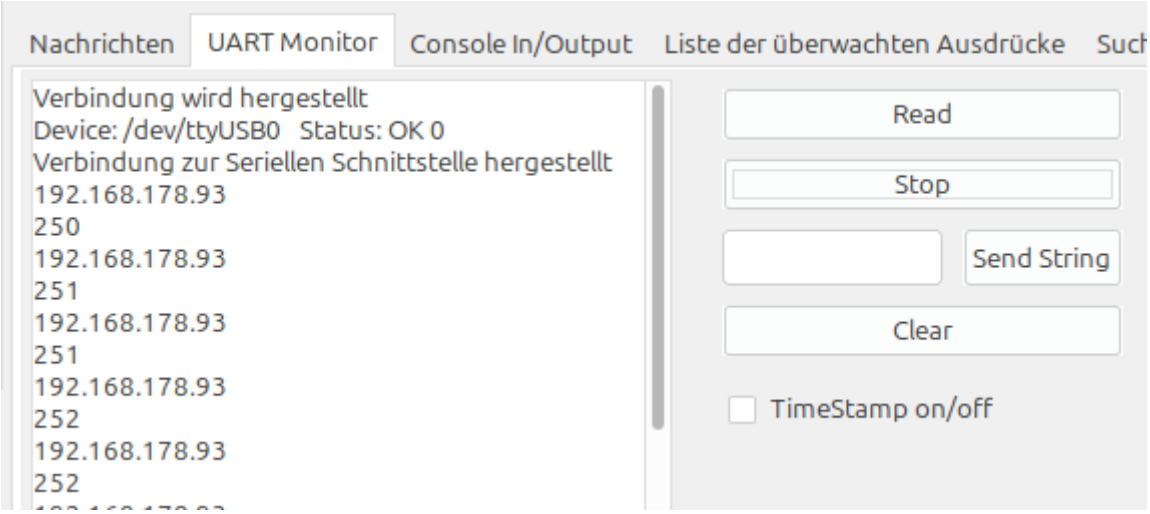
```
program udp_lesen_WIZ;

{$mode objfpc}{$H+}

uses
  fmem,
  SysUtils,
  lwip_sockets,
  task,
  esp_log2,
  portmacro,
  lwip_inet,
  lwip_def,
  wificonnect2,
  Laz_ESP;

const
  TAG = 'First_WIZ';
  HOST_IP_ADDR = '192.168.178.93'; // anpassen, das ist die IP des Gerätes an
das gesendet wird
  PORT = 38899;   //Dieser Port muss für die WIZ Geräte verwendet werden!

  AP_NAME  = //'DEINE_SSID';
  PWD      = //'DEIN_Netzwerkschlüssel';

  const payload_get  : PChar = '{"method": "getPower"}';


var
  payload       : PChar;

 function ExtractPower(json: PChar; out value: Integer): Boolean;
 var
   p: PChar;
   neg: Boolean;
 begin
    Result := False;
    value := 0;

    //Erwarteter json: {"method":"getPower","env":"pro","result":
{"power":62207}}
    p := StrPos(json, '"power":');
    if p = nil then
      Exit;

    Inc(p, 8); // hinter "power":

    // whitespace überspringen
    while (p^ = ' ') do
      Inc(p);

    // negatives Vorzeichen
    neg := False;
```

```
    if p^ = '-' then
    begin
      neg := True;
      Inc(p);
    end;

    // nur Zahlen
    if not (p^ in ['0'..'9']) then
      Exit;

    // Zahl parsen
    while (p^ in ['0'..'9']) do
    begin
      value := value * 10 + (Ord(p^) - Ord('0'));
      Inc(p);
    end;

    //von milliWatt nach Watt
    Value := round(Value / 1000);

    if neg then
      value := -value;

    Result := True;
 end;


procedure udp_client_task(pvParameters: Pointer);
var
  rx_buffer : array[0..127] of char;
  addr_str  : array[0..127] of char;
  addr_family : Integer;
  ip_protocol : Integer;

  destAddr : sockaddr_in;
  sourceAddr : sockaddr_in;
  sock : Integer;
  err : Integer;
  len : Integer;
  socklen : socklen_t;

  Watt : integer;

begin
  while True do
  begin
    FillChar(destAddr, SizeOf(destAddr), 0);

    destAddr.sin_addr.s_addr := inet_addr(HOST_IP_ADDR);
    destAddr.sin_family := AF_INET;
    destAddr.sin_port := lwip_htons(PORT);

    addr_family := AF_INET;
    ip_protocol := IPPROTO_IP;

    sock := lwip_socket(addr_family, SOCK_DGRAM, ip_protocol);

    if sock < 0 then
    begin
      ESP_LOGE(TAG, 'Unable to create socket',[]);
      break;
```

```
end;

ESP_LOGI(TAG, 'Socket created',[]);



while True do
begin

  err := lwip_sendto(
    sock,
    payload,
    strlen(payload),
    0,
    @destAddr,
    SizeOf(destAddr)
  );

  if err < 0 then
  begin
    ESP_LOGE(TAG, 'Error occurred during sending, %d',[Err]);
    break;
  end;

  socklen := SizeOf(sourceAddr);

  len := lwip_recvfrom(
    sock,
    @rx_buffer,
    SizeOf(rx_buffer) -1,
    0,
    @sourceAddr,
    @socklen
  );

  if len < 0 then
  begin
    ESP_LOGE(TAG, 'recvfrom failed, %d',[Len]);
    break;
  end
  else
  begin
    rx_buffer[len] := #0; //Hier wird Null terminiert
    ExtractPower(PChar(@rx_buffer[0]),Watt);
    writeln(Watt);

    inet_ntoa_r(sourceAddr.sin_addr, @addr_str, SizeOf(addr_str)-1);
    writeln(addr_str);

  end;

  vTaskDelay(5000 div portTICK_PERIOD_MS);

end;

if sock <> -1 then
begin
  ESP_LOGE(TAG, 'Shutting down socket and restarting... ,%d',[sock]);
  lwip_shutdown(sock, 0);
  lwip_close(sock);
end;
```

```
  end;

  vTaskDelete(nil);
end;

begin
  SerialBegin(9600);

  connectWifiAP(AP_NAME, PWD);

  Payload := payload_get;

  xTaskCreate(
    @udp_client_task,
    PChar('udp_client'),
    4096,
    nil,
    5,
    nil
  );

  while True do
  begin
   sleep(100);
  end;

end.
```

Ausgabe:

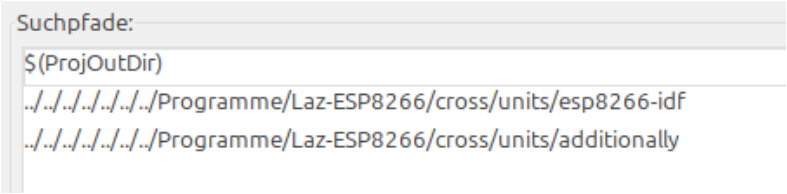


Das Programm 17_PDW_auslagern

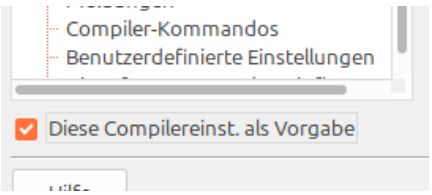
Im Programm 17 habe ich mein Passwort und den Netzwerkschlüssel in eine separate Inc-Datei ausgelagert. Das hat für mich den Vorteil das ich es nicht bei jedem Demo vor dem veröffentlichen löschen muss. Die Inc Datei habe ich in gitignore hinterlegt so das diese außen vor bleibt. Wer dies auch machen möchte braucht nur einen Texteditor öffnen und dort

```
const
 AP_NAME  = 'DEINE_SSID';
 PWD      = 'DEIN_Netzwerkschlüssel';
```

eingeben. Natürlich müssen die richtigen Daten eingetragen werden! Dann die Datei als PWD.inc abspeichern (einfach als Erweiterung inc eingeben). Man kann die Inc-Datei in den Projektordener kopieren das es funktioniert. Ich habe die Datei nach /home/bernd/Programme/Laz-ESP8266/cross/units/additionally kopiert. Damit Lazarus die Inc Datei dort findet muss man den Ordner in den Einstellungen angeben: Projekt, Projekteinstellungen, Pfade. Bei Include-Dateien auf die … klicken und den Pfad hinzufügen.



Möchte man das dies für alle neuen Projekte gleich so hinterlegt ist muss man bei „Diese Compilereinst. als Vorgabe“ einen Haken setzen und mit OK verlassen.



Nach den uses fügt man einfach `{$Include PWD.inc}` ein!

```
program project1;

uses
fmem,
laz_esp,
task,
portmacro,
wificonnect2;

{$Include PWD.inc}

begin
SerialBegin(9600);
connectWifiAP(AP_NAME, PWD);
repeat
 vTaskDelay(1000 div portTICK_PERIOD_MS);
until false ;
end.
```

Das Programm 18_Zeitserver

Im Programm 18 holt sich der ESP8266 das aktuelle Datum und die Zeit von einem Zeitserver. Gefunden habe ich diese Möglichkeit im Github Repository von ccrause im Beispiel „boreholepump“
https://github.com/ccrause/fpc-esp-freertos/tree/master/examples/boreholepump
Die benötigten zusätzlichen Units (time_server.pas,c_time.pas, esp_sntp.pas) fand ich in der IDF für den ESP32. Ich habe diese Dateien nach /home/bernd/Programme/Laz-ESP8266/cross/units/additionally kopiert. Den ursprünglichen Namen timeunit musste ich umbennen da es zu einem Konflikt mit einer bestehenden Datei kam.

Diese drei Methoden habe ich noch hinzugefügt:

```
procedure DateTimeAsString_de(out s: shortstring);
procedure DateAsString_de(out s: shortstring);
procedure TimeAsString_de(out s: shortstring);
```

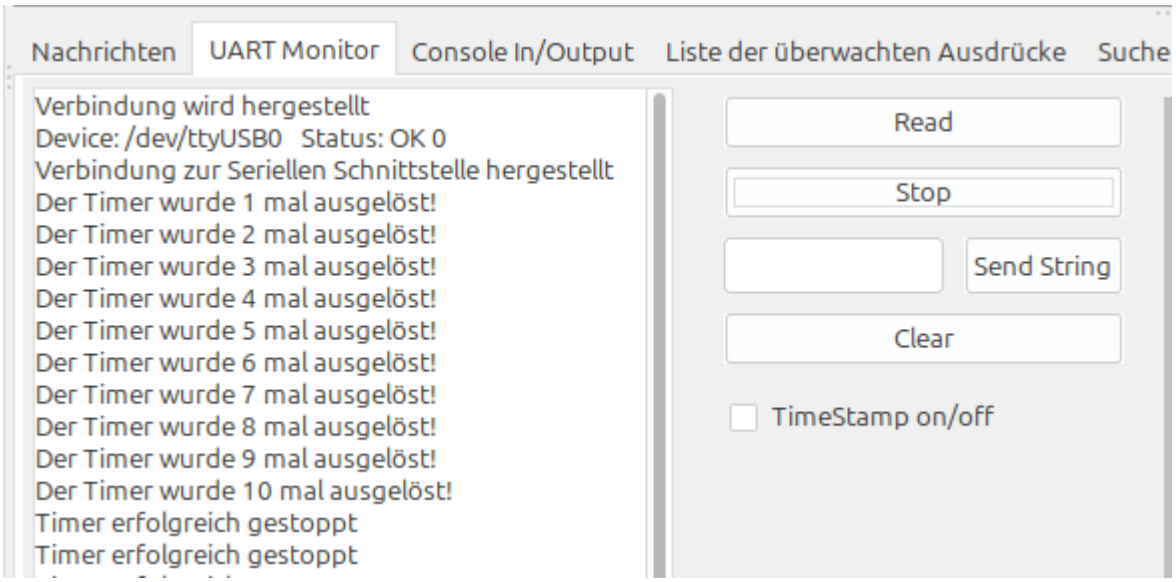
```
program project1;
uses
fmem,
laz_esp,
task,
portmacro,
wificonnect2,
time_server,
esp_sntp;

{$Include PWD.inc}

var
 s    : shortstring;
 Stat : Tsntp_sync_status;

begin
SerialBegin(9600);
connectWifiAP(AP_NAME, PWD);
//warten bis Wifi verbunden ist
repeat
 sleep(500);
 writeln('Wait for Wifi ......');
until stationConnected = true;
initTime;
//warten bis sntpCallback eintrifft
repeat
 sleep (100);
 Stat := sntp_get_sync_status;
until Stat = SNTP_SYNC_STATUS_COMPLETED;

repeat
 currentTimeAsString(s);
 writeln('Orginal: ',s);
 DateTimeAsString_de(s);
 writeln('Datum und Zeit: ',s);
 DateAsString_de(s);
 writeln('Datum: ',s);
 TimeAsString_de(s);
 writeln('Zeit: ',s);
 vTaskDelay(1000 div portTICK_PERIOD_MS);
until false ;
end.
```

Das Programm 19_MyTimer_01

Im Programm 19 wird ein Timer erzeugt und gestartet. Er feuert alle 3 Sekunden. Nachdem der Timmer 10x ausgelöst hat wird er beendet und freigegeben.
https://www.freertos.org/media/2025/FreeRTOS_Reference_Manual_V8.2.1.pdf

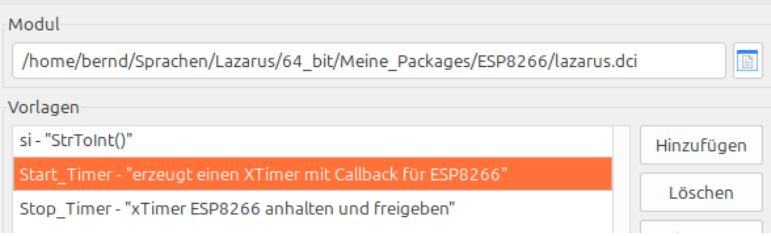
```
program esp8266_timer;

{$mode objfpc}{$H+}

uses
  fmem,
  timers,
  projdefs,
  laz_esp;

{$include freertosconfig.inc}
const
  TIMER_PERIOD = 3000; // ms also 3 Sekunden

var
  MyTimer: TTimerHandle;
  i      : integer;

procedure MyTimerCallback(xTimer: TTimerHandle);
begin
  inc(i);
  writeln('Der Timer wurde ' ,i,' mal ausgelöst!');
end;

procedure Start_Timer;
begin
  // Timer erstellen  MyTimer ist das Handle auf den Timer!
  MyTimer := xTimerCreate('MyTimer', // Name des Timers
                          pdMS_TO_TICKS(TIMER_PERIOD),// Periode in Ticks
                          pdTRUE,          // pdTRUE = wiederholend, pdFALSE =
einmalig
                          nil,             // Timer-ID optional
                          @MyTimerCallback);//Callback-Funktion

  if MyTimer = nil then
  begin
    writeln('TIMER', 'Timer konnte nicht erstellt werden');
    Exit;
  end;

  // Timer starten
  if xTimerStart(MyTimer, 0) <> pdPASS then
    writeln('TIMER', 'Timer konnte nicht gestartet werden');
end;

procedure Stopp_Timer;
begin
 // Timer stoppen
 if xTimerStop(MyTimer, 0) <> pdPASS then
  writeln('Timer konnte nicht gestoppt werden')
 else
  begin
   writeln('Timer erfolgreich gestoppt');
```

```
   //Wenn der Timer einmalig ist, kann man ihn auch löschen, um Speicher
freizugeben.
   xTimerDelete(MyTimer, 0);
  end;


end;

begin
  i := 0;
  SerialBegin(9600);
  Start_Timer;

 repeat
  sleep(100);
  if i = 10 then Stopp_Timer;
 until false ;
end.
```



Das Programm 20_MyTimer_02

Hier starte ich zwei Timer und fange die Nachrichten in der Callback-Funktion ab. Um die Timerfunktion einfacher nutzen zu können habe ich diese in die Codevervollständigung eingetragen.




```
program project1;

uses
  fmem, timers, projdefs,
  laz_esp;

var
  FirstTimer  : TTimerHandle;
  SEcondTimer : TTimerHandle;

procedure TimerCallback(xTimerHandle: TTimerHandle);
begin
 if xTimerHandle = FirstTimer then
   writeln('Message from Firsttimer');
 if xTimerHandle = SecondTimer then
   writeln('Message from Secondtimer');
end;

procedure Start_Timer(out xTimerHandle: TTimerHandle;Interval:LongInt);
begin
 XTimerHandle := xTimerCreate('aTimer', // Name des Timers
                              pdMS_TO_TICKS(Interval),// Periode in Ticks
                              pdTRUE,           // pdTRUE = wiederholend, pdFALSE = einmalig
                              nil,              // Timer-ID optional
                              @TimerCallback);//Callback-Funktion

 if xTimerHandle = nil then
  begin
   writeln('The timer could not be created');
   Exit;
  end;

// Timer starten
 if xTimerStart(xTimerHandle, 0) <> pdPASS then //um den Start des Timers zu verzögern die 0 ersetzen
  writeln('The timer could not be started');
end;

begin
 SerialBegin(9600);
 Start_Timer(FirstTimer,1000);
 sleep(500);
 Start_Timer(SecondTimer,2000);

 repeat
  sleep(100);
 until false ;
end.
```

Queue:

Eine Queue (Warteschlange) dient dazu, Daten sicher zwischen Tasks (Threads) auszutauschen. Eine Queue funktioniert nach dem Prinzip FIFO (First In, First Out). Queues sind Thread-sicher → keine Race Conditions (gleichzeitiger Zugriff auf die selben Daten).
https://www.freertos.org/Documentation/02-Kernel/02-Kernel-features/02-Queues-mutexes-and-semaphores/01-Queues

Record erstellen
Queue erstellen:

```
var
MyQueue: QueueHandle_t;

MyQueue := xQueueCreate(5, SizeOf(TSensorData));
```

Record senden:

```
var
  Data: TSensorData;
begin
  Data.Temperature := 25;
  Data.Humidity := 60;

  if xQueueSend(MyQueue, @Data, portMAX_DELAY) <> pdTRUE then
  begin
    Writeln('Queue konnte nicht gesendet werden');
  end;

  //alternativ:
  //xQueueOverwrite(MyQueue, @Data);
end;
```

Record Empfangen:

```
var
  Received: TSensorData;
begin
  if xQueueReceive(MyQueue, @Received, portMAX_DELAY) = pdTRUE then
  begin
    // Zugriff auf Daten
    // Received.Temperature
    // Received.Humidity
  end;
end;
```

portMAX_DELAY kann blockieren, Vorteil kein Wert geht verloren. Alternative:

```
const
WaitMS = 50;
TimeoutTicks = pdMS_TO_TICKS(WaitMS);

if xQueueSend(MyQueue, @Data, TimeoutTicks) <> pdTRUE then ……
```

Setzt die Queue komplett zurück (alle Elemente werden entfernt):

```
xQueueReset(queueHandle);
```

alle Einträge auslesen bis die Queue leer ist:

```
while xQueueReceive(queueHandle, @item, 0) = pdTRUE do
begin
// Elemente bearbeiten
end;
```

Hier enthält die Queue immer nur den neuesten Wert:

```
xQueueOverwrite(queue, @value);  Achtung bei: MyQueue := xQueueCreate(1, SizeOf(TSensorData));
```

Mutex:

Wenn du im RTOS SDK für den ESP8266 gleichzeitig auf denselben Speicherbereich schreibst und liest, braucht man einen Mutex (gegenseitiger Ausschluss), damit nicht zwei Tasks gleichzeitig zugreifen. Mein Problem war die Anzeige im Label, die ab und zu komische Zeichen ausgab. PChar zeigt nur auf Speicher – der ist nicht threadsicher. Du musst den Zugriff darauf schützen.

https://www.freertos.org/Documentation/02-Kernel/04-API-references/10-Semaphore-and-Mutexes/12-xSemaphoreTake

Beispiel:

```
var
Chunk4: PChar;
ChunkMutex: TsemaphoreHandle;

//Mutex erstellen (einmal beim Init)

ChunkMutex := xSemaphoreCreateMutex();

//Schreiben (Thread/Task 1)

if xSemaphoreTake(ChunkMutex, portMAX_DELAY) = pdTRUE then

begin

  StrPCopy(Chunk4, 'Neue Daten');

  xSemaphoreGive(ChunkMutex);

end;

//Lesen (Thread/Task 2)

if xSemaphoreTake(ChunkMutex, portMAX_DELAY) = pdTRUE then

begin

  WriteLn(StrPas(Chunk4));

  xSemaphoreGive(ChunkMutex);

end;
```

Das Programm 21_MyFirstHttpServer_05

Im Programm 21 sende und empfange ich die UDP Botschaften in einem extra Task der alle 5 Sekunden läuft. Der Task befindet sich in einer separaten Unit WifiUdp. Die Verarbeitung der erhaltenen Botschaften passiert in einem Timer. Der Timer startet mit einem Versatzt von 5 Sekunden und besitzt ein Intervall von 1 Sekunde. Der Austausch der Werte passiert über eine Warteschlange (Queue) um den gleichzeitigen Zugriff (Race Conditions) auf Variablen zu verhindern. Um feststellen zu können wann ein Button der Webseite gedrückt wurde habe ich einen zusätzlichen Handler eingefügt.

Im Http-Teil:

```
<form method="GET" action="/pushed">
<input type="submit" name="action" value="Ein" …...
<input type="submit" name="action" value="Aus" …...
```

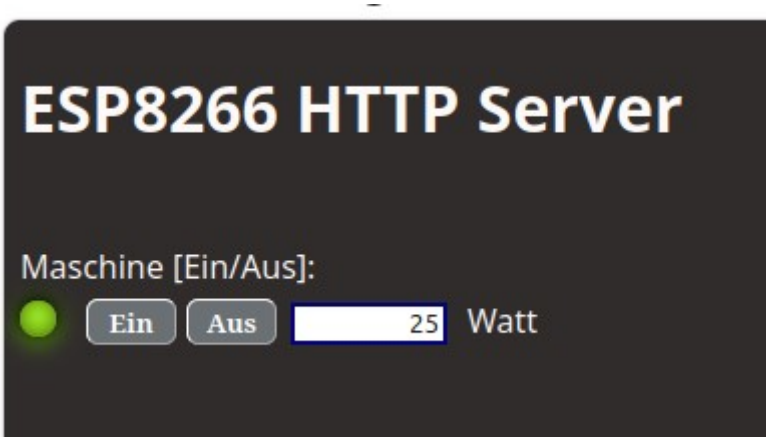
Die neue Funktion:

```
function WIZ_get_handler(req: Phttpd_req): Tesp_err;
……
httpd_resp_set_status(req, '303 See Other');  //setzt auf neue URL
httpd_resp_set_hdr(req, 'Location', '/');     //neue URL
…….
```

und noch registrieren:

```
with WIZUriHandlerConfig do
  begin
    uri       := '/pushed';
    method    := HTTP_GET;
    handler   := @WIZ_get_handler;
    user_ctx  := nil;
  end;
httpd_register_uri_handler(server, @WIZUriHandlerConfig);
```

Sicherlich lässt sich noch vieles verbessern aber (bei mir) läuft das Programm stabil, ich kann die Funksteckdose schalten und den momentanen Verbrauch ablesen.



```
program project1;

uses
  fmem,
  laz_esp,
  wificonnect2,
  portmacro,
  timers,
  projdefs,
  wifiudp,
  queue,
  esp_http_server,
  esp_err,
  portable,
  http_parser,
  semphr;

{$Include PWD.inc}

const
  Remote_IP_ADDR = '192.168.178.102'; // anpassen, das ist die IP des Gerätes an das gesendet wird
  PORT = 38899;   //Dieser Port muss für die WIZ Geräte verwendet werden!

  Payload_on   : PChar = '{"method":"setState","params":{"state":true}}';  //schaltet ein
  Payload_off  : PChar = '{"method":"setState","params":{"state":false}}'; //schaltet aus


var
  Timer1Handle : TTimerHandle;
  Chunk1       : PChar;
  Chunk2       : PChar;
  Chunk3       : PChar;
  Chunk4       : PChar;
  Chunk5       : PChar;
  ChunkMutex   : TSemaphoreHandle;
  WIZ1         : boolean;


function ExtractPower(json: PChar; out value: Integer): Boolean;
var
 pStart, pEnd : Integer;
 Watt,i       : Integer;
 valueStr     : shortstring;
begin
 Result := False;
 value := 0;

 pStart := Pos('"power":', json);
 if pStart > 0 then
  begin
   pStart := pStart + Length('"power":');

   // Ende der Zahl finden
   pEnd := pStart;
   while (pEnd <= Length(json)) and (json[pEnd] in ['0'..'9']) do
    Inc(pEnd);

   valueStr := Copy(json, pStart, pEnd - pStart+1);
   val(valueStr,Watt,i);
  end
 else
 begin
  WriteLn('Power nicht gefunden');
  exit;
 end;

 //von milliWatt nach Watt
  Value := round(Watt / 1000);
  Result := True;
end;

procedure TimerCallback(xTimerHandle: TTimerHandle);
var Received_udp : Tesp_udp;
    i, Watt      : integer;
```

```
    s            : shortstring;

begin

 if xQueueReceive(QueueHandle, @Received_Udp, 0) = pdTRUE then
  begin
    i := Pos('getPower',Received_Udp.Buffer0);
    if i <> 0 then
     begin
      ExtractPower(Received_Udp.Buffer0,Watt);
      s := inttostr(Watt);
      if xSemaphoreTake(ChunkMutex, portMAX_DELAY) = pdTRUE then
       begin
        StrToPChar(s,Chunk4);
        xSemaphoreGive(ChunkMutex);
       end;
      end;
    i := Pos('getPilot',Received_Udp.Buffer1);
    if i <> 0 then
     begin
      i := Pos('true',Received_Udp.Buffer1);
      if i <> 0 then WIZ1 := true
      else WIZ1 := false;
     end;
  end;
end;

procedure Start_Timer(out xTimerHandle: TTimerHandle;Interval:LongInt);
begin
xTimerHandle := xTimerCreate('Timer1', // Name des Timers
                             pdMS_TO_TICKS(Interval),// Periode in Ticks
                             pdTRUE,          // pdTRUE = wiederholend, pdFALSE = einmalig
                             nil,             // Timer-ID optional
                             @TimerCallback);//Callback-Funktion

if xTimerHandle = nil then
 begin
  writeln('The timer could not be created');
  Exit;
 end;

// Timer starten
if xTimerStart(xTimerHandle,5000) <> pdPASS then   //Timer startet um 5000 versetzt!
 writeln('The timer could not be started');
end;

procedure CreateWebpage;
begin
 Chunk1:= '<!DOCTYPE html><html>'+
           '<head>'+
            '<title>ESP8266 HTTP Server</title>'+
            '<meta http-equiv="refresh" content=20>'+   //Webseite aktualisiert sich alle 20
Sekunden
            '<style>'+
             'body {font-family: sans-serif;background-color:#302c2c}'+
             '.led-green {'+
             'margin-top: 6px;'+
             'width: 18px;'+
             'height: 18px;'+
             'background-color: greenyellow;'+
             'border-radius: 50%;'+
             'box-shadow: #000 0 -1px 7px 1px, inset #460 0 -1px 9px, #7D0 0 2px 12px;'+
             'float:left;}'+
             '.led-gray {'+
             'margin-top: 6px;'+
             'width: 18px;'+
             'height: 18px;'+
             'background-color: #C0C0C0;'+
             'border-radius: 50%;'+
             'box-shadow: #000 0 -1px 7px 1px, inset #9A9 0 -1px 9px, #969 0 2px 14px;'+
             'float:left;}'+
            '</style>'+
           '</head>'+
             '<form method="GET" action="/pushed">'+
             '<h1><font color="snow">ESP8266 HTTP Server</font></h1>'+
             '<br><font color="snow">Maschine [Ein/Aus]:</font></br>';
```

```
  if WIZ1 then Chunk2 := '<div class="led-green"></div>';

  if not WIZ1 then Chunk2 := '<div class="led-gray"></div>';

  Chunk3 :=  '<style>input{color:white}</style>'+
             '<input type="submit" name="action" value="Ein" style=''Background-color: #6b7074;'+
             'font-family:Bradley Hand ITC; font-size: 24; font-weight: bold;width: 45px;'+
             'border-radius: 6px;border-width: 1px;border-color: snow;margin-top: 5px;margin-
left:15px;''>'+

             '<input type="submit" name="action" value="Aus" style=''Background-color: #6b7074;'+
             'font-family:Bradley Hand ITC; font-size: 24; font-weight: bold;width: 45px;'+
             'border-radius: 6px;border-width: 1px;border-color: snow;margin-left: 5px;''>'+

             '<input type="text" name="WIZ1.1" value="';

 //Chunk4 :=   wird in der TimerCallback gesetzt

 Chunk5 :=   '" size="9" style=''border-color:rgb(3, 3, 125);'+
             'border-width: 2px;border-style: solid;text-align: right;margin-left: 7px;'+
             'background-color:rgb(255, 255, 255);color:black;width: 70px;'' readonly>'+

             '<label style= "margin-left: 10px;color : snow">Watt</label>'+

            '</form>'+
           '</html>';
end;

function main_get_handler(req: Phttpd_req): Tesp_err;

begin
 Result := ESP_FAIL;
 //writeln('Main_GetHandler');
 CreateWebpage;
 if xSemaphoreTake(ChunkMutex, portMAX_DELAY) = pdTRUE then
 begin
  httpd_resp_send_chunk(req, Chunk1, length(Chunk1));
  httpd_resp_send_chunk(req, Chunk2, length(Chunk2));
  httpd_resp_send_chunk(req, Chunk3, length(Chunk3));
  httpd_resp_send_chunk(req, Chunk4, length(Chunk4));
  httpd_resp_send_chunk(req, Chunk5, length(Chunk5));
  // Signal finish of chunks
  httpd_resp_send_chunk(req, nil, 0);
  result := ESP_OK;
  xSemaphoreGive(ChunkMutex);
 end;
end;


function WIZ_get_handler(req: Phttpd_req): Tesp_err;
var
  buf: PChar;
  buf_len: uint32;
  i : integer;
begin
 Result := ESP_FAIL;
 //writeln('Form Handler');
 buf_len := httpd_req_get_hdr_value_len(req, 'Host') + 1;
 if (buf_len > 1) then
 begin
  buf := pvPortMalloc(buf_len);
  if (httpd_req_get_hdr_value_str(req, 'Host', buf, buf_len) = ESP_OK) then
   writeln('Found header => Host: ', buf);
  vPortFree(buf);
 end;

 buf_len := httpd_req_get_url_query_len(req) +1;
  if buf_len > 1 then
  begin
   buf := pvPortMalloc(buf_len);
   if httpd_req_get_url_query_str(req, buf,buf_len) = ESP_OK then
   writeln('Query: ', buf);

   i := Pos('Ein',buf);
   if i <> 0 then
   begin
    Wiz1 := true;
```

```
    esp_udp.Payload := Payload_on;
    //writeln('WIZ1 ist ein');
   end;
   i := Pos('Aus',buf);
   if i <> 0 then
   begin
    Wiz1 := false;
    esp_udp.Payload := Payload_off;
    //writeln('WIZ1 ist aus');
   end;

   vPortFree(buf);
  end;

 // Signal finish of chunks
 httpd_resp_set_status(req, '303 See Other');  //setzt auf neue URL
 httpd_resp_set_hdr(req, 'Location', '/');     //neue URL
 httpd_resp_send_chunk(req, nil, 0);
 result := ESP_OK;
end;

function start_webserver: Thttpd_handle;
var
  server: Thttpd_handle;
  config: Thttpd_config;
  mainUriHandlerConfig: Thttpd_uri;
  WIZUriHandlerConfig: Thttpd_uri;

begin
  config := HTTPD_DEFAULT_CONFIG();

  with mainUriHandlerConfig do
  begin
    uri       := '/';
    method    := HTTP_GET;
    handler   := @main_get_handler;
    user_ctx  := nil;
  end;

  with WIZUriHandlerConfig do
  begin
    uri       := '/pushed';
    method    := HTTP_GET;
    handler   := @WIZ_get_handler;
    user_ctx  := nil;
  end;

  writeln('Starting server on port: ', config.server_port);
  if (httpd_start(@server, @config) = ESP_OK) then
  begin
    // Set URI handlers
    writeln('Registering URI handler');
    httpd_register_uri_handler(server, @mainUriHandlerConfig);
    httpd_register_uri_handler(server, @WIZUriHandlerConfig);
    result := server;
  end
  else
  begin
    result := nil;
    writeln('### Failed to start httpd');
  end;
end;



begin

 SerialBegin(9600);

 ChunkMutex := xSemaphoreCreateMutex();
 Chunk4 := '0';

 connectWifiAP(AP_NAME,PWD);
 repeat
  writeln('Start Wifi ...');
 until stationConnected = true;
```

```
 UDP_Send(Remote_IP_ADDR,Port,P1);

 Start_Timer(Timer1Handle,1000);

 writeln('Starting web server...');
 start_webserver;
 repeat
  sleep(100);
 until false ;
end.
```

```
unit WifiUdp;

{$mode ObjFPC}{$H+}

interface

uses
  task,
  portmacro,
  lwip_sockets,
  lwip_inet,
  lwip_def,
  queue, projdefs;

type
 Tesp_udp = record
  RemoteIP   : PChar;
  RemotePort : Word;
  Payload    : PChar;
  Buffer0    : PChar;
  Buffer1    : PChar;
  RecvIP     : PChar;
  err        : Integer;

 end;

type TJSONS = (J1,J2);

const
 P1  : PChar = '{"method": "getPower"}';   //liefert den momentanen Verbrauch in mW
 P2  : PChar = '{"method":"getPilot","params":{}}';   //Liefert den Schaltzustand ein/aus


var
 esp_udp        : Tesp_udp;
 QueueHandle    : TQueueHandle;
 timeout        : timeval;

procedure UDP_Send(RemoteIP : PChar; RemotePort : Word; Message : PChar);

implementation

var
 TAG  : PChar;
 JSON : TJSONS;

procedure ChangeJSON;
begin
 JSON := TJSONS((ord(JSON) + 1) mod 2);
 //writeln(ord(json));
 case ord(JSON) of
  0 : esp_udp.Payload := P1;
  1 : esp_udp.Payload := P2;
 end;
end;

procedure Send(pvParameters: Pointer);cdecl;
var
  rx_buffer : array[0..127] of char;
  addr_str  : array[0..127] of char;
  addr_family : Integer;
  ip_protocol : Integer;
```

```
  destAddr : sockaddr_in;
  sourceAddr : sockaddr_in;
  sock : Integer;

  len : Integer;
  socklen : socklen_t;
begin
  while True do
  begin
    FillChar(destAddr, SizeOf(destAddr), 0);
    FillChar(destAddr, SizeOf(sourceAddr), 0);

    timeout.tv_sec := 3;   // 3 Sekunden warten auf Antwort
    timeout.tv_usec := 0;

    destAddr.sin_addr.s_addr := inet_addr(esp_udp.RemoteIP);
    destAddr.sin_family := AF_INET;
    destAddr.sin_port := lwip_htons(esp_udp.RemotePort);

    addr_family := AF_INET;
    ip_protocol := IPPROTO_IP;

    sock := lwip_socket(addr_family, SOCK_DGRAM, ip_protocol);
    //falls keine Antwort kommt abbrechen timeout
    lwip_setsockopt(sock, SOL_SOCKET, SO_RCVTIMEO, @timeout, SizeOf(timeout));
    if sock < 0 then
    begin
      writeln(TAG,' , Unable to create socket');
      break;
    end;

    //Writeln(TAG, ' , Socked created');

    while True do
    begin
      esp_udp.err := lwip_sendto(
      sock,
      esp_udp.Payload,
      strlen(esp_udp.Payload),
      0,
      @destAddr,
      SizeOf(destAddr)
      );

      if esp_udp.err < 0 then
      begin
        writeln (TAG, ' , Error occurred during sending , Fehler: ', esp_udp.err);
        break;
      end;

      //writeln(TAG, ' , Message sent');

      socklen := SizeOf(sourceAddr);

      len := lwip_recvfrom(
        sock,
        @rx_buffer,
        SizeOf(rx_buffer) -1,
        0,
        @sourceAddr,
        @socklen
      );

      if len < 0 then
      begin
        writeln(TAG,' , recv failed');
        esp_udp.err := -1;
        break;
      end
      else
      begin
        if destAddr.sin_addr.s_addr = sourceAddr.sin_addr.s_addr then
         begin
          //writeln('Antwort erhalten');

          rx_buffer[len] := #0;
```

```
          if ord(json) = 0 then
           begin
            esp_udp.Buffer0 := rx_buffer;
            //writeln(0,rx_buffer);
           end;
          if ord(json) = 1 then
           begin
            esp_udp.Buffer1 := rx_buffer;
            //writeln(1, rx_buffer);
           end;

          inet_ntoa_r(sourceAddr.sin_addr, @addr_str, SizeOf(addr_str));
          esp_udp.RecvIP := PChar(addr_str);

          if xQueueSend(QueueHandle, @esp_udp, portMAX_DELAY) <> pdTRUE then
           begin
            writeln('Queue could not send!');
           end;
         end;
      end;
      ChangeJSON;
      vTaskDelay(5000 div portTICK_PERIOD_MS);

    end;

    if sock <> -1 then
    begin
      //writeln(TAG, ' , Shutting down socket and restarting... , Socket: ',sock);
      lwip_shutdown(sock, 0);
      lwip_close(sock);
    end;

  end;
  vTaskDelete(nil);
end;


procedure UDP_Send(RemoteIP : PChar; RemotePort : Word; Message : PChar);
begin
 esp_udp.RemoteIP   := RemoteIP;
 esp_udp.RemotePort := RemotePort;
 esp_udp.Payload    := Message;
 esp_udp.err        := -1;
 TAG  := 'UDPSend';

 QueueHandle := xQueueCreate(1, SizeOf(Tesp_udp));
 xTaskCreate(@Send, 'UDP_Send', 4096, nil, 4, nil);
end;

end.
```

SPIFFS – mit Dateien umgehen

SPIFFS steht für “Serial Peripheral Interface Flash File System”. SPIFFS ist dafür gedacht, Daten direkt im Flash-Speicher eines ESP8266 zu speichern und zu verwalten.

Um es nutzen zu können müssen einige Punkte verändert werden. Zunächst benötigt man eine neue Partitionstabelle. Um diese erzeugen zu können benötigt man eine *.csv Datei. Meine erste funktionierende Tabelle sieht so aus:

```
# Name,   Type, SubType, Offset,   Size
nvs,      data, nvs,     0x9000,   0x4000
phy_init, data, phy,     0xd000,   0x1000
factory,  app,  factory, 0x10000,  0x90000
spiffs,   data, spiffs,  0xA0000,  0x160000
```

NVS = Non-Volatile Storage, Zweck: Speichert dauerhafte Daten (z. B. WLAN-Zugangsdaten, Einstellungen)

PHY = Physical Layer (WLAN-Funkparameter), Zweck: Enthält Kalibrierungsdaten für das WLAN

factory =Hauptprogramm (Firmware), Zweck: Hier liegt dein Programm (Sketch) Größe 0,59MB

SPIFFS-Dateisystem, Zweck: Speicher für Dateien Größe 1,44MB

Ich habe mich im Nachgang noch einmal mit der Erzeugung einer Partitionstabelle befasst und möchte hier meine Erkenntnisse einschieben:

Ich habe mir je eine kleine Tabelle gemacht und Offset und Size in Dezimal angehängt um den Offset leichter berechnen zu können. Size muss immer ein vielfaches von 4096 sein!

Bei meinem 1. Versuch belege ich insgesamt rund 2 MB

1. Spiffs: Summe 2052096 = 2,05 MB

# Name,	Type,	SubType,	Offset,	Size	Offset	Size
nvs,	data,	nvs,	0x9000,	0x4000	36864	6384
phy_init,	data,	phy,	0xd000,	0x1000	53248	4096
factory,	app,	factory,	0x10000,	0x90000	65536	589824
spiffs,	data,	spiffs,	0xA0000,	0x160000	655360	1441792

Offset im Flashbefehl 0xA0000

Bei meinem nächsten Versuch habe ich mir die Werte für nvs und phy_init aus der singleapp.csv im ESP8266_RTOS_SDK-Release 3.4 geholt und die gesamt Größe auf ca. 3,4 MB erhöht.

2. Versuch Summe 3436544 = 3,4 MB

```
# Name,    Type, SubType, Offset,   Size,        Offset      Size
nvs,       data, nvs,     0x9000,  0x6000,       36864       24576       Werte aus singleapp.csv
phy_init,  data, phy,     0xf000,  0x1000,       61440       4096        Werte aus singleapp.csv
factory,   app,  factory, 0x10000, 0x1E0000,     65536       1966080      = 480 x 4096
spiffs,    data, spiffs,  0x1F0000,0x160000,     2031616     1441792      = 352 x 4096
```

2. CSV 3,4 MB

```
# Name,    Type, SubType, Offset,   Size,
nvs,       data, nvs,     0x9000,   0x6000,
phy_init,  data, phy,     0xf000,   0x1000,
factory,   app,  factory, 0x10000,  0x1E0000,
spiffs,    data, spiffs,  0x1F0000, 0x160000,
```

Je nach Anforderung lässt sich so der Speicherbedarf anpassen.

https://docs.espressif.com/projects/esp8266-rtos-sdk/en/latest/api-guides/partition-tables.html

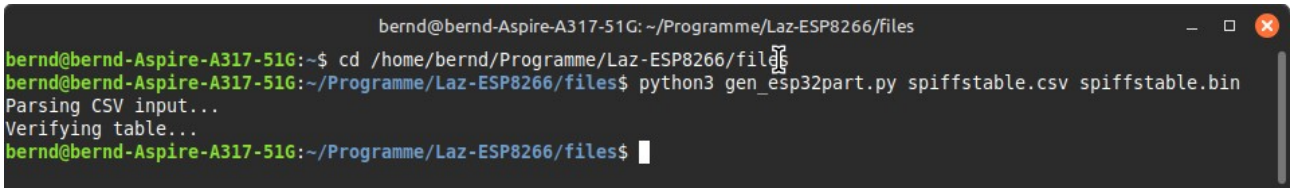
Ab hier geht es wieder weiter mit dem erzeugen einer Partitionstabelle.

Am besten legt man einen leeren Ordner an (keine Leerzeichen und Umlaute im Pfad!) und speichert dort die csv ab. Ich habe dies unter /home/bernd/Programme/Laz-ESP8266/files mit dem Dateinamen spiffstable.csv gemacht.

Als nächstes benötigt man die Datei gen_esp32part.py. Diese findet man im ESP8266_RTOS_SDK-Release 3.4. Von dort kopieren und in den neu erstellten Ordner einfügen.

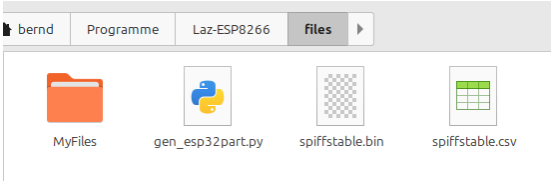
Dann ein Terminal öffnen mit cd zu dem Ordner navigieren und

`python3 gen_esp32part.py spiffstable.csv spiffstable.bin` eingeben.



Die Datei spiffstable.bin sollte nun vorhanden sein. Das ist die neue Partitionstabelle.

Als nächstes muss man aus einem Ordner ein Image erzeugen in das die Dateien gespeichert werden können. Zunächst legt man einen neuen Ordner an. Ich habe in MyFiles genannt.



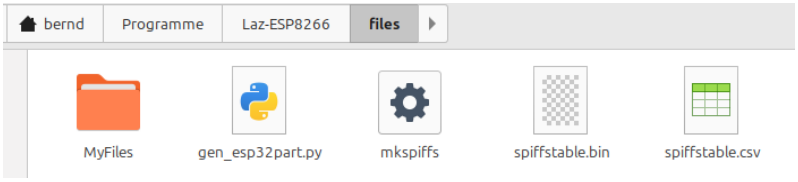
Übrigens wenn man Dateien in den Ordner MyFiles kopiert werden diese in das Image übertragen und auf den ESP geflasht.

Dann benötigt man das Programm mkspiffs. Dieses macht aus dem leeren Ordner ein Image das der ESP8266 lesen kann. Wer mir Arduino arbeitet wird es dort finden. Ansonsten kann man es sich hier clonen und selber bauen:

```
git clone --recursive https://github.com/igrr/mkspiffs.git

cd mkspiffs

make
```

```
bernd@bernd-Aspire-A317-51G:~$ git clone --recursive https://github.com/igrr/mkspiffs.git /home/bernd/Programme/Laz-ESP8266/mkspiffs
Klone nach '/home/bernd/Programme/Laz-ESP8266/mkspiffs'...
remote: Enumerating objects: 328, done.
remote: Counting objects: 100% (3/3), done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 328 (delta 0), reused 2 (delta 0), pack-reused 325 (from 1)
Empfange Objekte: 100% (328/328), 216.28 KiB | 3.79 MiB/s, fertig.
Löse Unterschiede auf: 100% (171/171), fertig.
Submodul 'spiffs' (https://github.com/pellepl/spiffs.git) für Pfad 'spiffs' in die Konfiguration eingetragen.
Klone nach '/home/bernd/Programme/Laz-ESP8266/mkspiffs/spiffs'...
remote: Enumerating objects: 1558, done.
remote: Counting objects: 100% (75/75), done.
remote: Compressing objects: 100% (43/43), done.
remote: Total 1558 (delta 37), reused 52 (delta 27), pack-reused 1483 (from 1)
Empfange Objekte: 100% (1558/1558), 1.08 MiB | 9.16 MiB/s, fertig.
Löse Unterschiede auf: 100% (1072/1072), fertig.
Submodul-Pfad 'spiffs': 'f5e26c4e933189593a71c6b82cda381a7b21e41c' ausgecheckt
bernd@bernd-Aspire-A317-51G:~$ cd /home/bernd/Programme/Laz-ESP8266/mkspiffs
bernd@bernd-Aspire-A317-51G:~/Programme/Laz-ESP8266/mkspiffs$ make
g++ -std=gnu++11 -Os -Wall   -Itclap -Iinclude -Ispiffs/src -I. -D VERSION=\"0.2.3-7-gf248296\" -D SPIFFS_VERSION=\"0.3.7-5-gf5e26c4\" -D BUILD_CONFIG=
\"\" -D BUILD_CONFIG_NAME=\"-generic\" -D __NO_INLINE__    -c -o main.o main.cpp
main.cpp: In function 'int actionUnpack()':
```

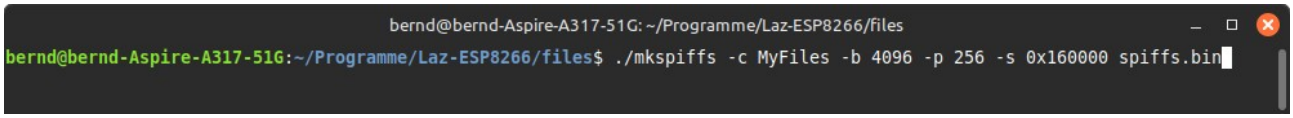
Nun das erzeugte mkspiffs in den Ordner Files kopieren:

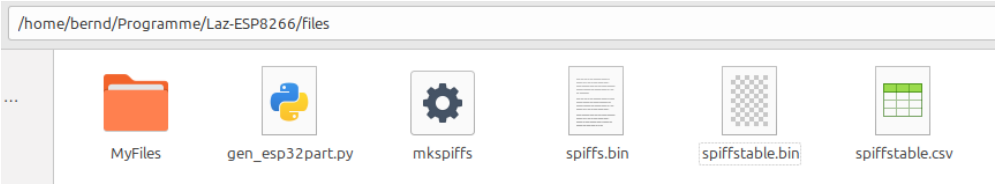


Und nun mit dem Befehl aus MyFiles ein Image mit dem Namen spiffs.bin erzeugen:

./mkspiffs -c MyFiles -b 4096 -p 256 -s 0x160000 spiffs.bin

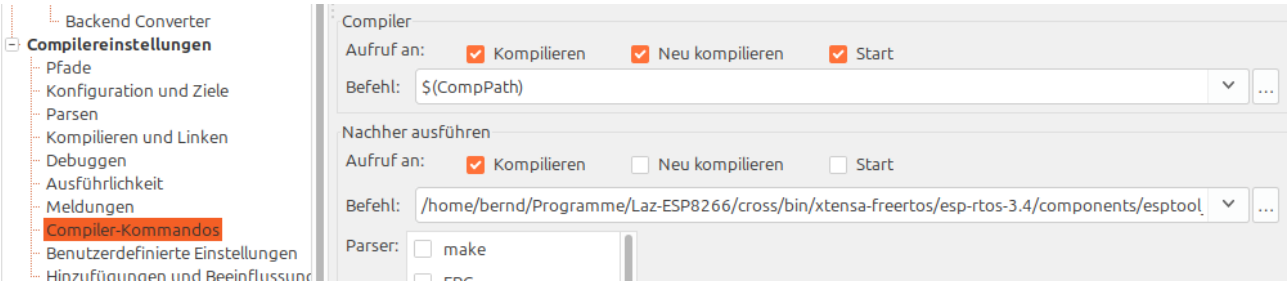
Wichtig: Die Größe muss identisch mit der Größe in der Partitionstabelle sein (hier 0x160000)





Nun die spiffs.bin und die spiffstable.bin nach /home/bernd/Programme/Laz-ESP8266/cross/lib/xtensa-freertos/lx106 kopieren.

Zuletzt muss noch der Flash-Befehl in Lazarus angepasst werden. Dazu Projekt, Projekteinstellungen, Compiler_Kommandos, Nachher ausführen, Befehl.




```
/home/bernd/Programme/Laz-ESP8266/cross/bin/xtensa-freertos/esp-rtos-3.4/components/esptool_py/
esptool/esptool.py --chip esp8266 --port "/dev/ttyUSB0" --baud 115200 --before "default_reset" --
after "hard_reset" write_flash -z --flash_mode "dio" --flash_freq "40m" --flash_size "4MB"   0x0
/home/bernd/Programme/Laz-ESP8266/cross/lib/xtensa-freertos/lx106/bootloader.bin 0x10000 $
(TargetFile).bin 0xA0000 /home/bernd/Programme/Laz-ESP8266/cross/lib/xtensa-freertos/lx106/
spiffstable.bin
```

Achtung die Pfade anpassen!!!

Das Programm 22_SpiffExample_01

Im ESP8266_RTOS_SDK-Release 3.4 findet man unter /home/bernd/Programme/Laz-ESP8266/ESP8266_RTOS_SDK-Release 3.4/examples/storage/spiffs/main das Programm spiffs_example_main.c. Dies war mein Anhaltspunkt für mein erstes funktionierendes Spiffs-Programm. Ich habe zunächst versucht die Pascal Methoden: AssignFile, Rewrite(f), CloseFile(f) usw. zuverwenden. Das habe ich leider nicht hinbekommen. Ich bin deshalb auf die C-Funktionen ausgewichen:

```
function fopen(filename, mode: PChar): PFILE; cdecl; external;

function fclose(f: PFILE): cint; cdecl; external;

function fprintf(f: PFILE; fmt: PChar): cint; cdecl; varargs; external;

function fgets(buf: PChar; size: cint; f: PFILE): PChar; cdecl; external;
```

```
fopen:

Öffnet eine Datei und liefert einen Dateizeiger (f) zurück
```

Öffnungsmodi:

"r" – Lesen (Datei muss existieren)

"w" – Schreiben (Datei wird neu erstellt)

"a" – Anhängen

"rb", "wb", "ab" – Binärmodi

"r+", "w+", "a+" – Lesen und Schreiben

fclose:
fclose schließt eine Datei, die zuvor mit fopen geöffnet wurde

fprintf:
schreibt formatierten Text in eine Datei

```
zum Beispiel: fprintf(f, 'Hallo Welt!%s', #10); fprintf(f, 'Zahl: %d%s', 42, #10); fprintf(f,
'Temperatur: %.2f°C%s', 23.456, #10);
```

fgets:
liest bis zu (maxlen-1) Zeichen aus einer Datei in einen Puffer und beendet den Text immer mit #0 (Nullterminator).

```
program SpiffExample;

{$linklib spiffs, static}

uses
  fmem,
  ctypes,
  esp_err,
  esp_log2,
  laz_esp,
  esp_spiffs;

var
  i : integer;


type
  PFILE = Pointer;

type
  Pesp_vfs_spiffs_conf_t = ^esp_vfs_spiffs_conf_t;
  esp_vfs_spiffs_conf_t = record
    base_path: PChar;
    partition_label: PChar;
    max_files: cint;
    format_if_mount_failed: boolean;
  end;

function fopen(filename, mode: PChar): PFILE; cdecl; external;
function fclose(f: PFILE): cint; cdecl; external;
function fprintf(f: PFILE; fmt: PChar): cint; cdecl; varargs; external;
function fgets(buf: PChar; size: cint; f: PFILE): PChar; cdecl; external;


procedure app_main;
var
  TAG: PChar = 'example';
  conf: esp_vfs_spiffs_conf_t;
  ret: esp_err_t;
  total, used: size_t;
  buf: array[0..63] of char;
  P : PChar;
  f : PFile;
begin
  ESP_LOGI(TAG,'%s',['Initializing SPIFFS']);

  conf.base_path := '/spiffs';
  conf.partition_label := nil;
  conf.max_files := 1;//5;
  conf.format_if_mount_failed := True;

  ret := esp_vfs_spiffs_register(@conf);

  if ret <> ESP_OK then
  begin
    if ret = ESP_FAIL then
      ESP_LOGE(TAG,'s%',['Failed to mount or format filesystem'])
    else if ret = ESP_ERR_NOT_FOUND then
      ESP_LOGE(TAG,'s%' ,['Failed to find SPIFFS partition'])
    else
      ESP_LOGE(TAG,'s%' ,[esp_err_to_name(ret)]);
    exit;
  end;

  total := 0;
  used := 0;

  ret := esp_spiffs_info(nil, @total, @used);
  if ret <> ESP_OK then
    ESP_LOGE(TAG,'%s' ,[esp_err_to_name(ret)])
  else
    writeln('[INFO] Partition size: total=', total, ' used=', used);


 { Datei schreiben }
  f := fopen('/spiffs/test.txt', 'w');
  if f = nil then
```

```
  begin
    ESP_LOGE(TAG,'%s',[ 'fopen write fehlgeschlagen']);
    exit;
  end;

  fprintf(f, 'Hallo Freunde von FreePascal',#10);
  fclose(f);

  { Datei lesen }
  f := fopen('/spiffs/test.txt', 'r');
  if f = nil then
  begin
    ESP_LOGE(TAG,'%s', ['fopen read fehlgeschlagen']);
    exit;
  end;

  if fgets(@buf[0], sizeof(buf), f) <> nil then
   begin
    P := @buf[0];
    ESP_LOGI(TAG, 'Gelesen: %s', [P]);
   end;
  fclose(f);

  esp_vfs_spiffs_unregister(nil);
  ESP_LOGI(TAG, '%s', ['SPIFFS unmounted']);
end;

begin
 SerialBegin(9600);
 //serielle Ausgabe etwas verzögern
 i := 100;
 repeat
  dec(i);
  writeln(i);
 until i = 0 ;

 app_main;

 repeat
  sleep(1000);
  writeln('Bin in der Loop angelangt');
 until false ;

end.
```

Das Programm 23_SpiffExample_02

Im Programm 23 habe ich zunächst die Spiffs-Befehle und die File-Befehle in je eine separate Unit ausgelagert.

Unit Spiffs

```
unit esp_spiffs;

{$mode objfpc}{$H+}

{$linklib spiffs, static}

interface

uses

  ctypes,

  esp_log2,

  esp_err;

type

  esp_err_t = cint;

  Psize_t = ^size_t;

  size_t = csize_t;

  Pesp_vfs_spiffs_conf_t = ^esp_vfs_spiffs_conf_t;

  esp_vfs_spiffs_conf_t = record

    base_path: PChar;

    partition_label: PChar;

    max_files: size_t;

    format_if_mount_failed: Boolean;

  end;

//Register and mount SPIFFS to VFS

function esp_vfs_spiffs_register(conf: Pesp_vfs_spiffs_conf_t): esp_err_t; cdecl; external;

//Unregister and unmount SPIFFS

function esp_vfs_spiffs_unregister(partition_label: PChar): esp_err_t; cdecl; external;

//Check if SPIFFS is mounted

function esp_spiffs_mounted(partition_label: PChar): Boolean; cdecl; external;

//Format SPIFFS partition

function esp_spiffs_format(partition_label: PChar): esp_err_t; cdecl; external;

//Get SPIFFS info

function esp_spiffs_info(partition_label: PChar;total_bytes: Psize_t;used_bytes: Psize_t):
esp_err_t; cdecl; external;

procedure RegisterSpiffs(MaxFiles:size_t;aPath, aTag : PChar);

procedure UnRegisterSpiffs(aTag :PChar);

implementation

procedure RegisterSpiffs(MaxFiles:size_t;aPath,aTag : PChar);

var
```

```
 conf: esp_vfs_spiffs_conf_t;
 ret: esp_err_t;
 total, used: size_t;
begin
 ESP_LOGI(aTAG,'%s',['Initializing SPIFFS']);

 conf.base_path              := aPath;
 conf.partition_label        := nil;
 conf.max_files              := MaxFiles;
 conf.format_if_mount_failed := True;
 total                       := 0;
 used                        := 0;

 ret := esp_vfs_spiffs_register(@conf);

 if ret <> ESP_OK then
  begin
   if ret = ESP_FAIL then
    ESP_LOGE(aTAG,'s%',['Failed to mount or format filesystem'])
   else if ret = ESP_ERR_NOT_FOUND then
    ESP_LOGE(aTAG,'s%' ,['Failed to find SPIFFS partition'])
   else
    ESP_LOGE(aTAG,'s%' ,[esp_err_to_name(ret)]);
    exit;
  end;

 ret := esp_spiffs_info(nil, @total, @used);
 if ret <> ESP_OK then
  ESP_LOGE(aTAG,'%s' ,[esp_err_to_name(ret)])
 else
  writeln('[INFO] Partition size: total=', total, ' used=', used);
end;
procedure UnRegisterSpiffs(aTag :PChar);
begin
 esp_vfs_spiffs_unregister(nil);
 ESP_LOGI(aTAG, '%s', ['SPIFFS unmounted']);
end;
end.
```

Unit FileIO

```
unit esp_fileIO;
interface
uses
 ctypes,
 esp_log2,
 laz_esp;
type
  PFILE = Pointer;
function fopen(filename, mode: PChar): PFILE; cdecl; external;
function fclose(f: PFILE): cint; cdecl; external;
function fprintf(f: PFILE; fmt: PChar): cint; cdecl; varargs; external;
function fgets(buf: PChar; size: cint; f: PFILE): PChar; cdecl; external;
function remove(path: PChar): cint; cdecl; external;
function unlink(path: PChar): cint; cdecl; external;
function FileExists(const aPath: PChar): boolean;
function FileWrite (const aPath,aText : PChar):boolean;
function FileAppend (const aPath,aText : PChar):boolean;
procedure FileRead (const aPath : PChar);
function FileReadLine(const aPath: PChar; out line: PChar): boolean;
function FileDelete(const aPath : PChar):boolean;
implementation
function FileExists(const aPath: PChar): boolean;
var
  f: PFILE;
begin
  f := fopen(aPath, 'r');
  if f <> nil then
  begin
    fclose(f);
    exit(True);
  end;
  Result := False;
end;

function FileWrite (const aPath,aText : PChar):boolean;
var
  f: PFILE;
begin
 f := fopen(aPath, 'w');
```

```
  if f = nil then
  begin
    ESP_LOGE('FileWrite','%s',[ 'fopen write failed']);
    exit(false);
  end;
 fprintf(f,'%s%s',aText,PChar(LineEnding));
 fclose(f);
 Result := True;
end;

function FileAppend (const aPath,aText : PChar):boolean;
var
  f: PFILE;
begin
 f := fopen(aPath, 'a');
  if f = nil then
  begin
    ESP_LOGE('FileAppend','%s',[ 'fopen write failed']);
    exit(false);
  end;
 fprintf(f, '%s%s', aText, PChar(LineEnding));
 fclose(f);
 Result := True;
end;

procedure FileRead (const aPath : PChar);
var
 f: PFILE;
 buf: array[0..255] of Char;
begin
  f := fopen(aPath, 'r');
  if f <> nil then
  begin
    while fgets(@buf[0], SizeOf(buf), f) <> nil do
     write (PChar(@buf[0]));
    fclose(f);
  end;
end;

function FileReadLine(const aPath: PChar; out line: PChar): boolean;
```

```
var
  f: PFILE;
  buffer: array[0..127] of char;
begin
  Result := False;
  line := '';

  f := fopen(aPath, 'r');
  if f = nil then exit;

  if fgets(@buffer[0], SizeOf(buffer), f) <> nil then
  begin
    line := PChar(@buffer[0]);
    Result := True;
  end;

  fclose(f);
end;

function FileDelete(const aPath : PChar):boolean;
var
  res: Integer;
  f    : PFILE;
begin
  if not FileExists(aPath) then
   exit(false);

  f := fopen(aPath, 'r');
  if f <> nil then
   fclose(f);

  res := remove(aPath);
  if res < 0 then
   exit(false)
  else
   Result := true;
 end;
end.
```

Ob dies der letzte Stand bleibt wird sich zeigen. Für das nachfolgende Programm ist es soweit okay. Im Programm 23 wird eine Datei angelegt und alle 10 Sekunden ein Eintrag hinein geschrieben. Alle 60 Sekunden wird die Datei gelesen und in der seriellen Schnittstelle ausgegeben. Bei einem Neustart wird die Datei (falls vorhanden) gelöscht. Nicht vergessen der Flash-Befehl muss angepasst werden!

```
program Spiffs02;
uses
  fmem,
  timers,
  projdefs,
  laz_esp,
  esp_spiffs,
  esp_fileIO;

var
 i           : Integer;
 TAG         : PChar = 'SpiffsExamle02';
 Path        : PChar = '/MyFolder';
 FilePath    : PChar = '/MyFolder/MyFile';
 aTimerHandle: TTimerHandle;
 temp1, temp2: PChar;

procedure TimerCallback(xTimerHandle: TTimerHandle);

begin
 if not FileExists(FilePath) then
  FileWrite(FilePath,'Das ist eine von mir erzeugte Datei')
 else
  begin
   inc(i);
   writeln('Eintrag: ',i,' wird geschrieben');
   temp1 := IntToPChar(i);
   temp2 := 'Dies ist Eintrag Nr.: ';
   FileAppend(FilePath,PChar(ConcatPChar(temp2,temp1)));

  end;
 if i mod 6 = 0 then
  begin
   if i <> 0 then writeln('Alle 60 Sekunden wird die Datei gelesen:');
```

```
   FileRead(FilePath);
  end;
end;
procedure Start_Timer(out xTimerHandle: TTimerHandle;Interval:LongInt);
begin
 XTimerHandle := xTimerCreate('Timer1', // Name des Timers
                              pdMS_TO_TICKS(Interval),// Periode in Ticks
                              pdTRUE,          // pdTRUE = wiederholend, pdFALSE = einmalig
                              nil,             // Timer-ID optional
                              @TimerCallback);//Callback-Funktion
 if xTimerHandle = nil then
  begin
   writeln('The timer could not be created');
   Exit;
  end;
// Timer starten
 if xTimerStart(xTimerHandle, 1000) <> pdPASS then
  writeln('The timer could not be started');
end;

begin
 SerialBegin(9600);
 //serielle Ausgabe etwas verzögern
 i := 50;
 repeat
  dec(i);
  writeln(i);
 until i = 0 ;

 RegisterSpiffs(5,Path,Tag);
 sleep(2000);//etwas warten bis spiffs registriert
 if FileDelete(FilePath) then writeln ('File deleted');
 Start_Timer(aTimerHandle,10000);

 repeat
  sleep(1000);
  writeln('Loop');
 until false ;
 UnRegisterSpiffs(Tag);
end.
```

Das Programm 24_Random

Um den Zufallszahlengenerator einfach nutzen zu können habe ich die Funktion

function getrandom(buf: pointer; buflen: PtrUInt; flags: cardinal): PtrInt;cdecl; external;

in die unit laz_ESP aufgenommen.

Das habe ich in den c Headern dazu gefunden:

Get random bytes from the kernel's random number generator

```
  Parameters:
     buf     - pointer to buffer to fill with random bytes
     buflen  - number of bytes to generate
     flags   - control flags (GRND_NONBLOCK, GRND_RANDOM)


   Returns:
     On success: number of bytes written to buf
     On error: -1 (and errno is set)
  const
  { Flags for getrandom }
  GRND_NONBLOCK = $0001;  { Do not block if entropy pool not initialized }
  GRND_RANDOM   = $0002;  { Use /dev/random instead of /dev/urandom }
```

Flag 0 ist offensichtlich der Standard. Die Wahl der Variable von buf bestimmt den Wertebereich (nimmt man byte hat man 0 – 255). Zusätzlich habe ich die Funktion Random und die Funktion GetRandomInRange erzeugt.

```
function Random: integer;
var
  buffer: integer;
  bytesRead: integer;
begin
 bytesRead := getrandom(@buffer, 4, 0);
 if bytesRead > 0 then
   Result := buffer
  else
   ESP_LOGE('Random','%s',['No random number found']);
end;
```

```
function GetRandomInRange(minVal, maxVal: integer): integer;
var
  buffer: LongWord;
  bytesRead: integer;
begin
  bytesRead := getrandom(@buffer, 4, 0);
  if bytesRead > 0 then
   Result := minVal + (buffer mod (maxVal - minVal + 1))
  else
   ESP_LOGE('GetRandomInRange','%s',['No random number found']);
end;



program testrandom;
uses
  fmem,
  laz_esp;
var
  buffer : integer;
  ret,i  : integer;

begin
 SerialBegin(9600);

 repeat
  sleep(500);

  ret := getrandom(@buffer, 4, 0);
  writeln('Anzahl Bytes gelesen: ',ret);
  i := buffer;
  writeln('getrandom: ',i);

  i:=Random;
  writeln('Random: ',i);

  i:=GetRandomInRange(-50,100);
  writeln('In Range -50-100: ',i);
 until false ;
end.
```